SERVICENOW – BASED EMPLOYEE LAPTOP REQUEST APPLICATION

Full Project Documentation

ServiceNow – Studio, Record Producer, Flow Designer, Approvals, Business Rules, ACLs, and Update Sets

Project Name	Employee Laptop Request Application
Platform	ServiceNow
Application Scope	Project Application
Primary Interface	Service Portal / Record Producer
Workflow	Service Catalog / Record Producer → Custom Table Record → Status Update → Manager Approval → Status Update / Email Notification

1.Project Overview :

This project puts in place an automated Employee Laptop Request management process in ServiceNow. Users submit hardware requests through the Service Catalog Record Producer, the request data is stored in a custom Laptop Request table, manager approval and status updates are initiated through Flow Designer, data policies are validated via Business Rules, and users receive automated status-based email notifications.

Project Lifecycle :

Phase 1: Requirement Analysis & Planning

Phase 2: Backend Development & Configuration

Phase 3: Workflow Development & Execution

Phase 4: Business Rule Creation, Security & Validation

Phase 5: Deployment, Final Presentation & Conclusion

Phase 6: Conclusion

Evidence Summary :

The implementation evidence in this document is based on the configured ServiceNow components and its embedded system screenshots. The screenshots are arranged under each respective project phase to illustrate custom table creation, form layout and relationship setup, Record Producer variable mappings, automated Flow Designer logic with manager approvals and email actions, Business Rule validation scripts, role-based ACL security configurations, and the completed Update Set XML export.

2. Phase 1: Requirement Analysis & Planning

Phase 1 concentrates on converting the manual employee laptop provisioning process into an automated, auditable ServiceNow solution by methodically analyzing key business requirements—including defining user roles (Requester, Manager, IT Fulfillment), establishing tiered approval levels based on hardware type (Basic vs. Advance), capturing essential request fields, and enforcing a one-request-per-employee-per-year validation rule. It sets out the architectural blueprint for the implementation by defining the Service Catalog intake form, designing the Flow Designer lifecycle from submission to asset delivery, setting up automated email notifications for milestone updates, and configuring operational dashboards to track fulfillment cycle times and hardware demand.

Business Objective

The main goal of this story is to set up a centralized, auditable system in ServiceNow that replaces manual, unverified laptop requests (such as emails and verbal approvals) with a standardized digital workflow. It enforces operational governance by differentiating requests into Basic and Advanced hardware tiers, preventing inventory churn through backend validation that restricts employees to one request per year, and promoting organizational accountability through automated manager approvals, real-time lifecycle tracking, and complete fulfillment visibility for IT and management teams.

Functional Scope

The functional scope defines an automated intake and governance framework within ServiceNow, enabling employees to submit standardized laptop requests via the Service Portal with captured employee details, categorized justifications, and designated hardware tiers (Basic for standard office tasks vs. Advance for technical workloads). The system offers transparent lifecycle tracking across Pending Approval, Approved, and Rejected states while enforcing a hard backend

policy restriction that limits each employee to a single laptop request per year to eliminate duplicate provisioning and protect inventory.

Record Producer Configuration

The Record Producer configuration offers a straightforward Service Portal intake form for employees to submit laptop requests, directly creating records in the custom Laptop Request table while enforcing role-based access, automatically triggering backend approvals, and running validation scripts to block duplicate requests within the same year.

Stakeholder Mapping

Field Label	Field Type	Target / Options	Purpose
Number	Auto Number	System Numbering	Unique identifier for tracking the request
Requester	Reference	sys_user	Employee submitting the laptop request
Employee Email	Email	Dynamic from sys_user	Contact email for status updates and notifications
Request Date	Date/Time	javascript:gs.nowDateTime()	Timestamp when the request was submitted
Short Description	String	Text Input	Brief summary of the laptop request
Manager	Reference	sys_user	Approving manager for the request workflow
Type of Request	Choice	Basic / Advance	Hardware tier classification
Status	Choice	Pending Approval / Approved / Rejected / Requested	Current lifecycle state of the request
Reason	Choice	Software Development / Personal Use / Other	Business justification for the request

Execution Roadmap

The Employee Laptop Request Management System updates IT asset provisioning by replacing manual emails and verbal requests with a centralized ServiceNow workflow that standardizes intake, enforces role-based hardware tiering (*Basic* vs. *Advance*), and protects inventory by automatically preventing duplicate requests within a one-year period.

Milestone	**Focus**	**Key Deliverables & Actions**
Milestone 1	Project Initialization & Update Set	Created and set HR Onboarding Application Update Set to **Current** to capture all development changes.
Milestone 2	Custom Application Setup	Built the scoped Project Application to isolate code, manage namespaces, and set access permissions.
Milestone 3	Table Design & Data Model	Created the Laptop Request Form table with auto-numbering, sys_user references, and choice fields.
Milestone 4	Business Logic & Rules	Implemented server-side Business Rules to enforce default values and block duplicate requests within a 1-year window.
Milestone 5	Testing & Automation Readiness	Conducted end-to-end functional testing, verified Update Set capture, and prepped the data model for Flow Designer integration.

3.Phase 2: Backend Development & Configuration

Phase 2 concentrates on building the core technical foundation in ServiceNow by configuring the custom tables, user roles, and catalog items, while implementing server-side business rules, notifications, and Flow Designer workflows to automate manager approvals, IT task assignments, and end-to-end request processing.

3.1.Data Architecture

1. Navigating to Local Update Sets

Before creating any custom configuration, you must navigate to the update set module to manage changes across environments.

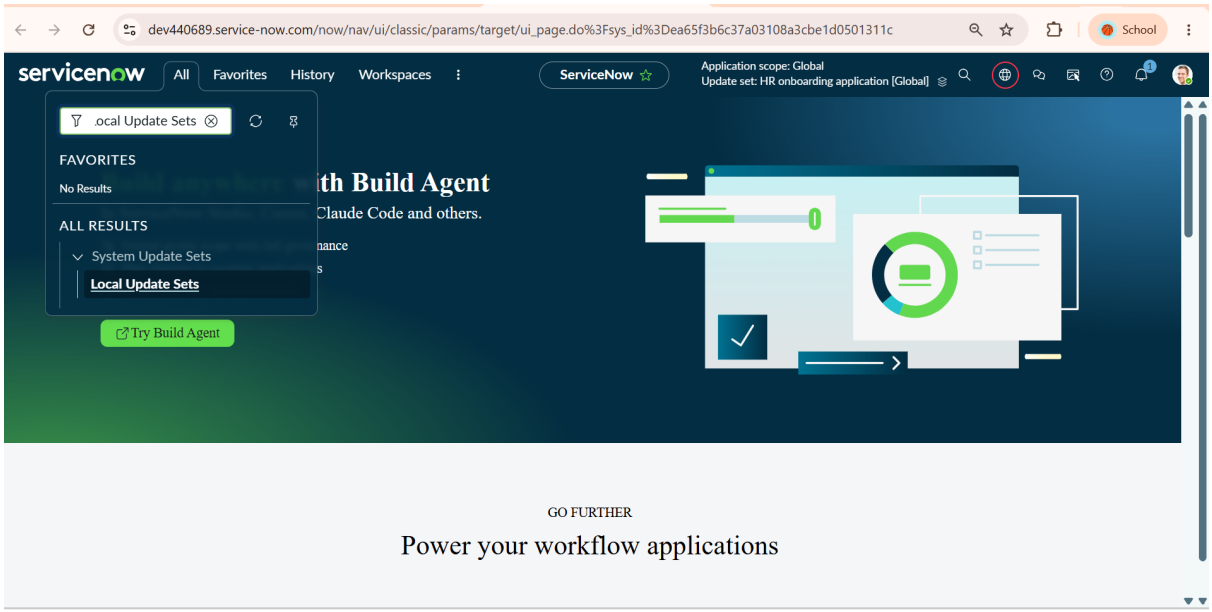


Figure 3.1.1 – Navigating to Local Update Sets via Application Navigator

2. Initiating a New Update Set

Open the update set table list to view existing update sets and start creating a dedicated container for your project changes.

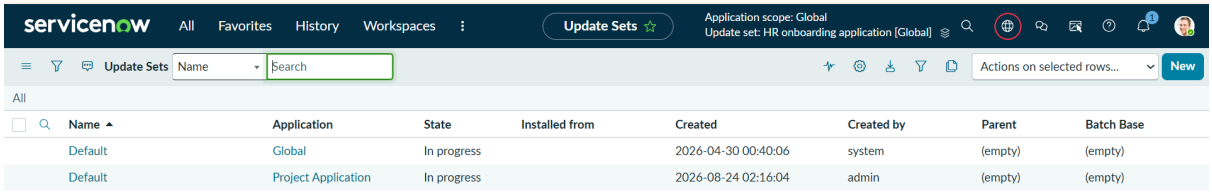


Figure 3.1.2 – System Update Sets List View and New Record Initiation

3. Configuring and Activating the Update Set

Specify the update set metadata so that all platform configurations are captured in this active scope.

The Update Set form filled with:

- **Name:** HR onboarding application
- **State:** In progress
- **Release date & Short description** filled out.

The top banner/header showing the active update set is set to **HR onboarding application** (verifying "Submit and Make Current").

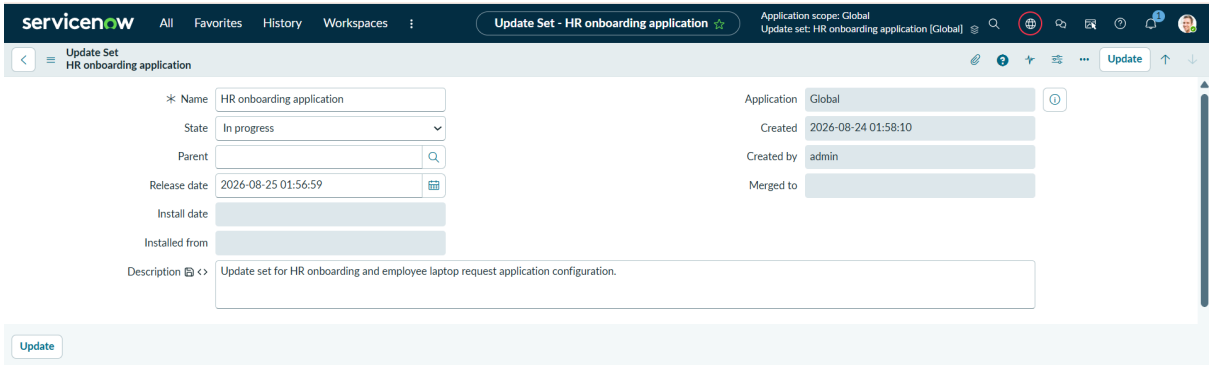


Figure 3.1.3 – HR Onboarding Application Update Set Configuration Form

4. Launching ServiceNow Studio

Open ServiceNow Studio the integrated development environment (IDE) used to build and manage custom scoped applications.

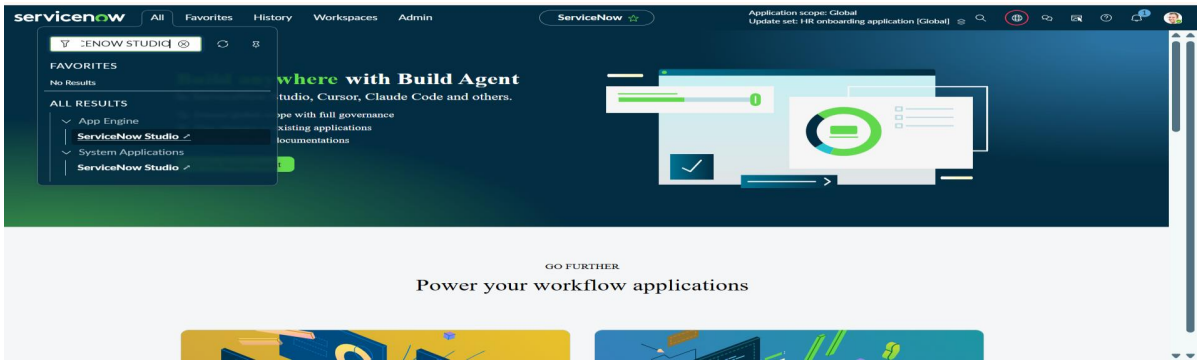


Figure 3.1.4 – Accessing ServiceNow Studio from All Menu

5. Selecting the Application Scope

Select the target application container to ensure all new database artifacts remain within the scoped application's namespace.

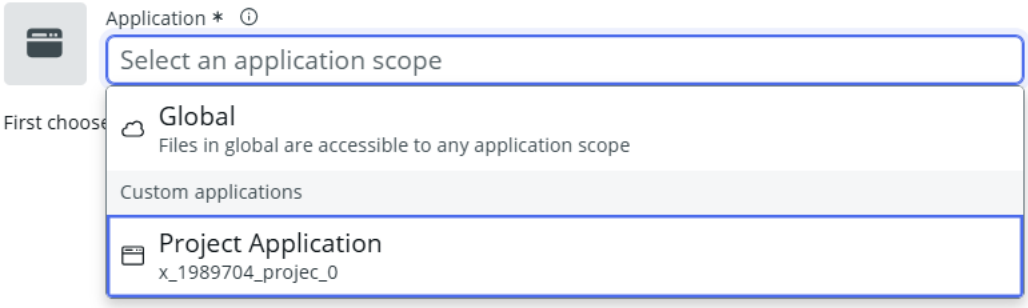


Figure 3.1.5 – Selecting Project Application Scope in Studio

6. Creating a New Table Artifact

Add a database object file to define the data model for the custom application.

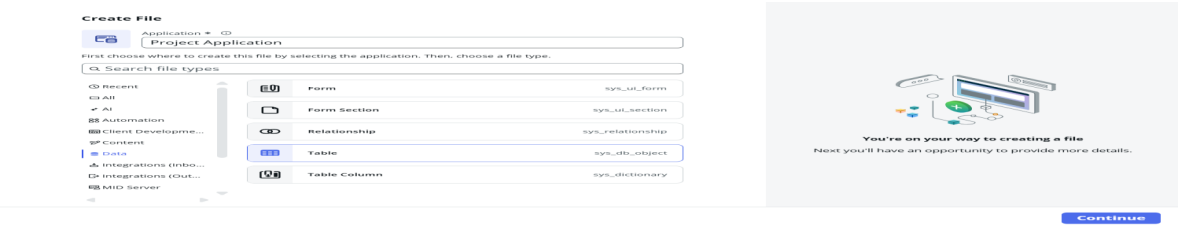


Figure 3.1.6 – Creating a New Table Data Artifact in Studio

7. Defining Schema Fields and Data Types

Configure the database columns, specifying appropriate field types (Reference to sys_user, Choice lists, Strings, and Date/Time).**Configured Fields:** Number,

Requester, Manager, Employee Email, Date, Short description, Type of Laptop, Status, Reason.

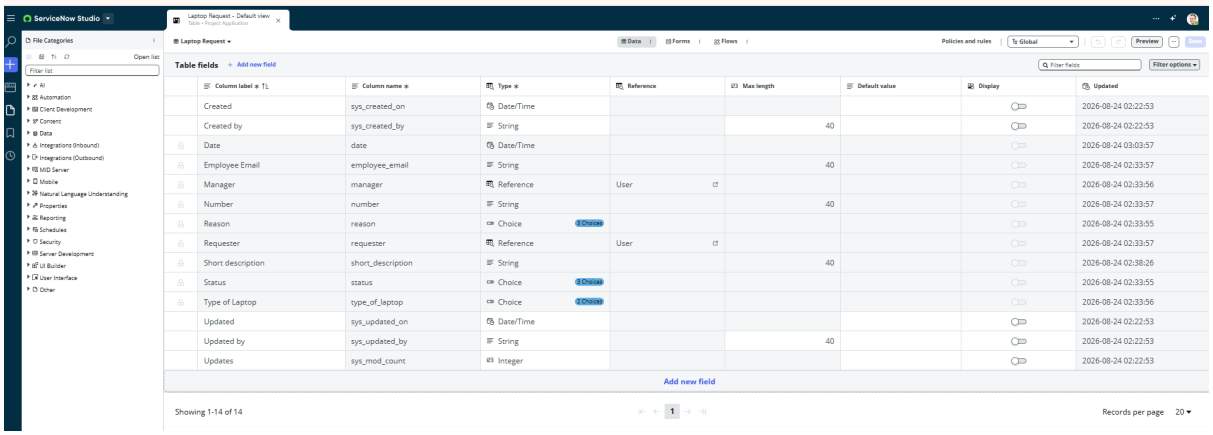


Figure 3.1.7 – Laptop Request Table Schema and Field Architecture Configuration

8. Form Layout Verification

Check the user interface form view to ensure the fields render correctly in a two-column split layout for end users.

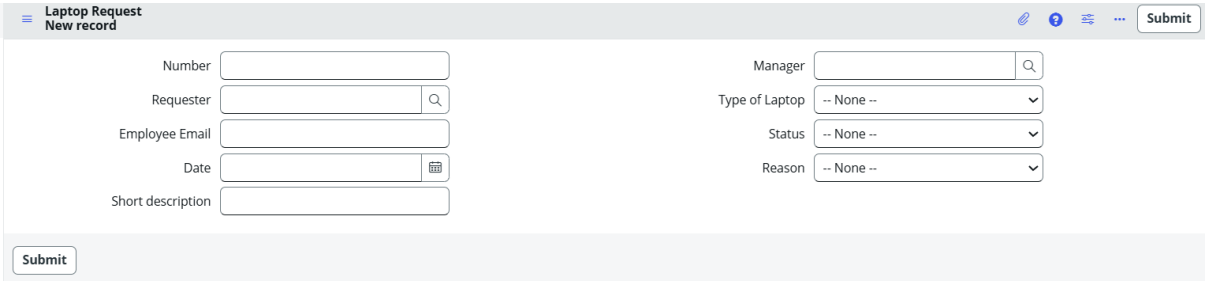


Figure 3.1.8 – Laptop Request Form Layout and UI Verification

3.2. Approval Table Relationship

In ServiceNow, standard approval engines rely on the centralized Approval table (sysapproval_approver) to track and manage authorizations across different processes. When developing a custom application, configuring a dedicated Relationship definition bridges your custom table (such as the Laptop Request form) to the global approval table without modifying base system schemas. By leveraging server-side query scripts, ServiceNow dynamically links and displays the corresponding approval records as an embedded related list on the request form, providing managers and stakeholders with end-to-end visibility and control over approval states.

1. Navigating to Relationships Module

To enable approval workflows for custom scoped tables like the Laptop Request form, you must define a custom relationship definition. This allows ServiceNow to dynamically link records from the core Approval table (sysapproval_approver) to your custom table based on matching criteria.

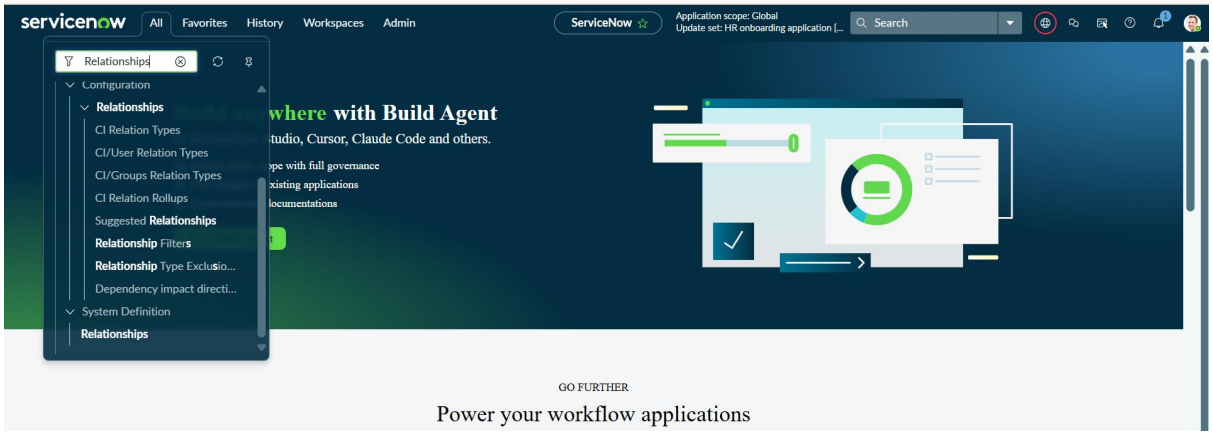


Figure 3.2.1 – Navigating to Relationships in System Definition

2. Configuring the Relationship and Query Script

Creating a custom relationship requires specifying what table the relationship applies to (Laptop Request) and what table it queries from (Approval [sysapproval_approver]). A server-side script using refineQuery maps the parent record's sys_id and table name to the approval record's document_id and source_table fields, ensuring only relevant approvals display on the specific form.

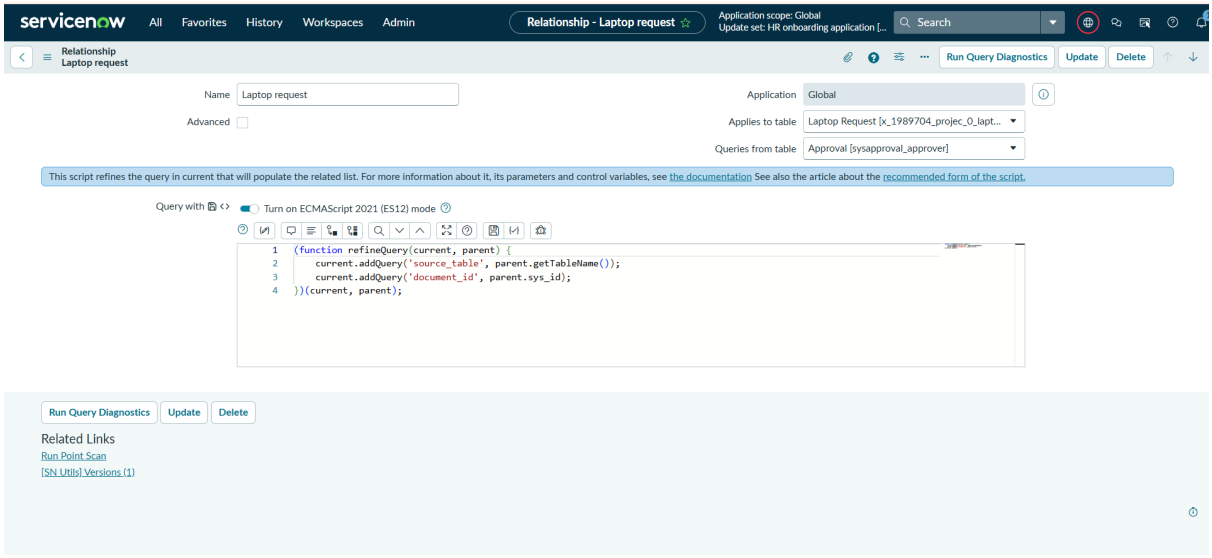


Figure 3.2.2 – Configuring Laptop Request Relationship and RefineQuery Script

3. Validating the Attached Approval Related List

Once the relationship is configured and added to the form layout via Configure > Related Lists, the platform dynamically renders the approval tracking section. This allows managers and administrators to view, track, and manage approval states (such as Requested, Approved, or Rejected) directly from the individual laptop request record.

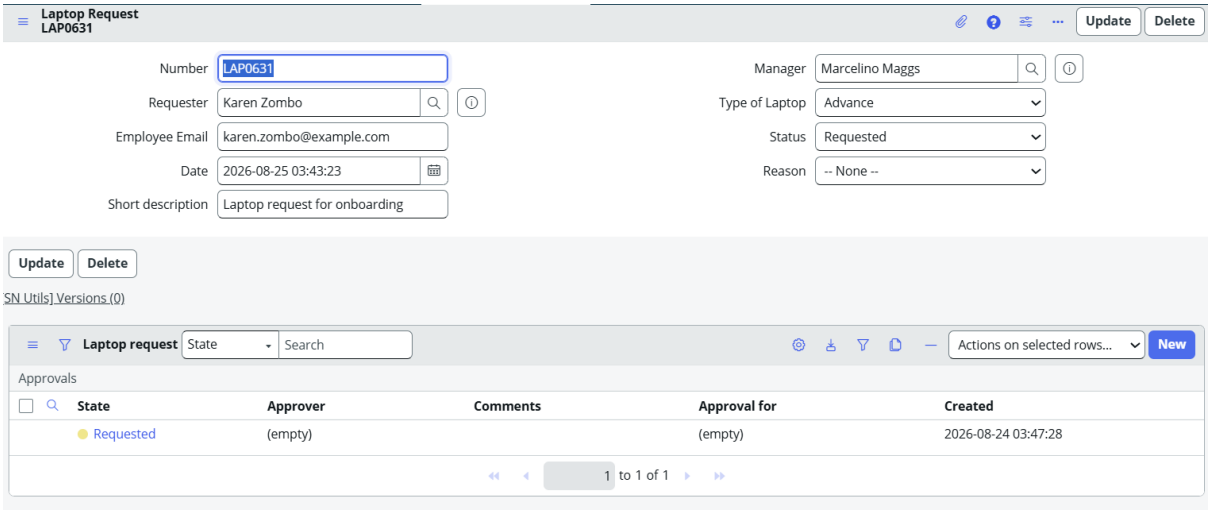


Figure 3.2.3 – Laptop Request Form with Attached Approval Related List View

3.3. Record Producer Configuration

In ServiceNow, a Record Producer serves as a catalog item that enables end users to log records directly into a target database table from the Service Portal or Service Catalog. Instead of giving users direct access to backend forms, Record Producers offer an easy-to-use intake form. By mapping user-facing questions (variables) directly to corresponding database columns (fields) in the custom Laptop Request table, the platform smoothly translates service catalog submissions into backend records while triggering underlying business workflows.

1. Navigating to Record Producers

Open the Catalog Definitions module within the Service Catalog application menu to view and build custom record creation interfaces.

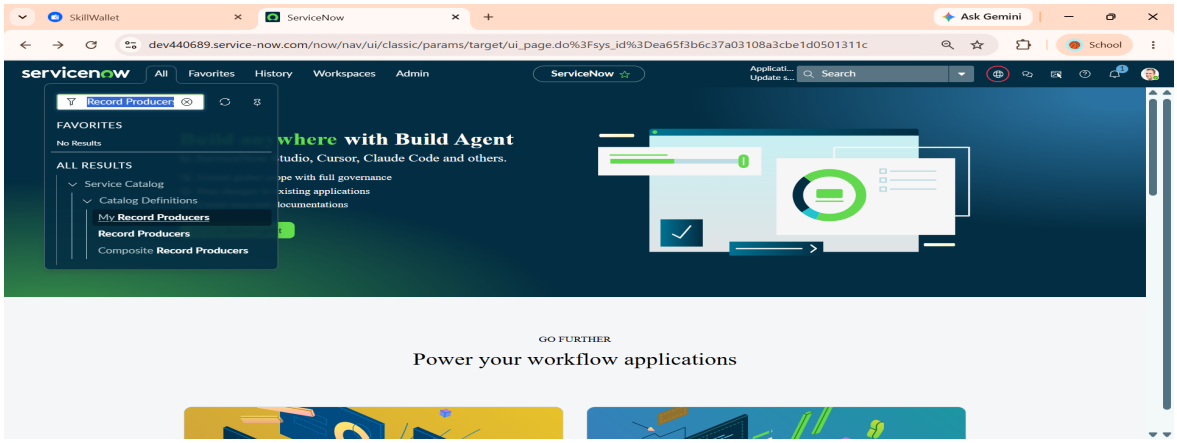


Figure 3.3.1 – Navigating to Record Producers under Service Catalog Definitions

2. Configuring the Record Producer Header & Target Table

Specify the core properties of the Record Producer by specifying the item name, linking it directly to the target scoped database table (x_1989704_projec_0_laptop_request), and writing user instructions/descriptions for the catalog view.

Key Fields:

- **Name:** Employee laptop request
- **Table name:** Laptop Request [x_1989704_projec_0_laptop_request]
- **Short description:** Laptop request submission form

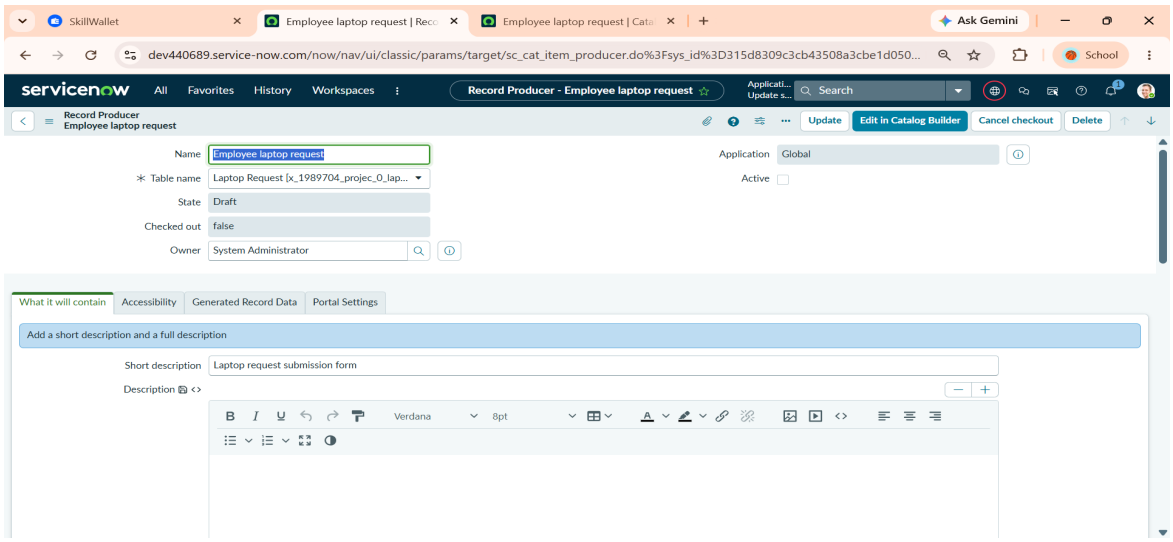


Figure 3.3.2 – Defining Record Producer Details and Target Table Association

3. Variable Definition and Field Mapping

Add catalog variables that prompt the user for input and map each variable directly to its corresponding backend table field to ensure submitted data populates correctly.

Configured Variables & Field Mappings:

Order	Question (User Prompt)	Target Field	Variable Type
100	What is Employee name?	requester	Reference
200	Employee mail Id	employee_email	Email
300	When did you want	date	Date/Time
400	Short description	short_description	Multi Line Text
500	Type of Laptop	type_of_laptop	Multiple Choice
600	Status	status	Multiple Choice
700	Reason	reason	Multiple Choice

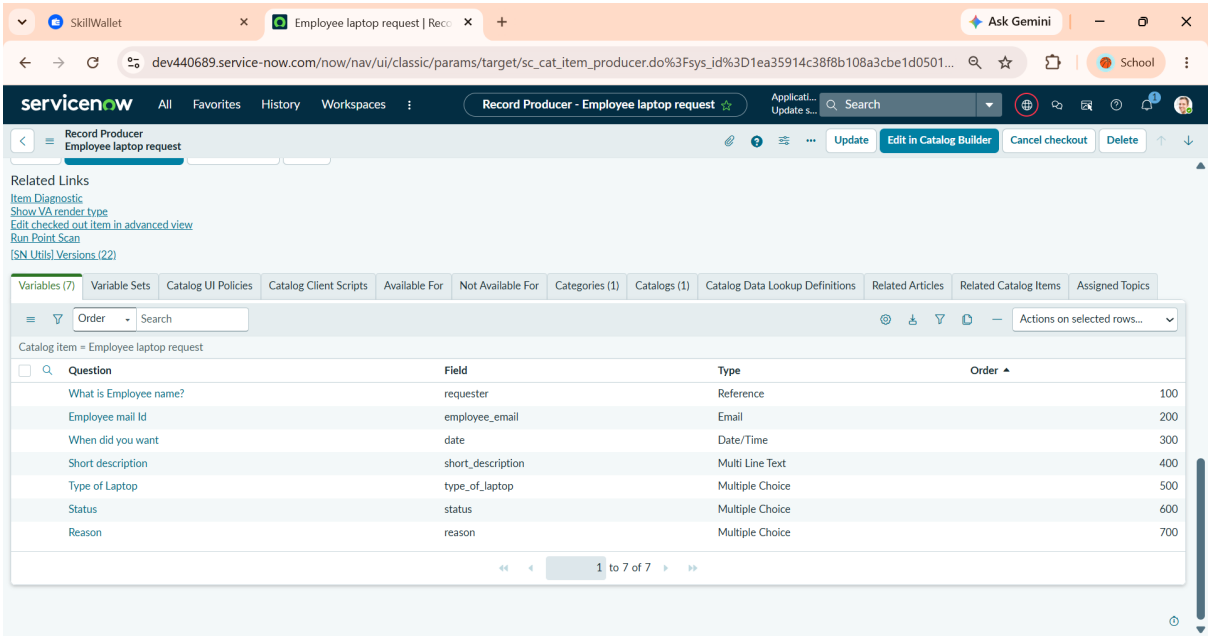


Figure 3.3.3 – Configuring Record Producer Variables and Direct Field Mapping

4. End-to-End Generated Record Validation

Confirm that submitting the Record Producer generates a new record in the custom Laptop Request table with all mapped values populated across both columns and the related approval structure attached. **Generated Record Data:** Number LA00231, Requester Antony Alldis, Manager Marcelino Maggs, Status Pending Approval, Reason Software Development.

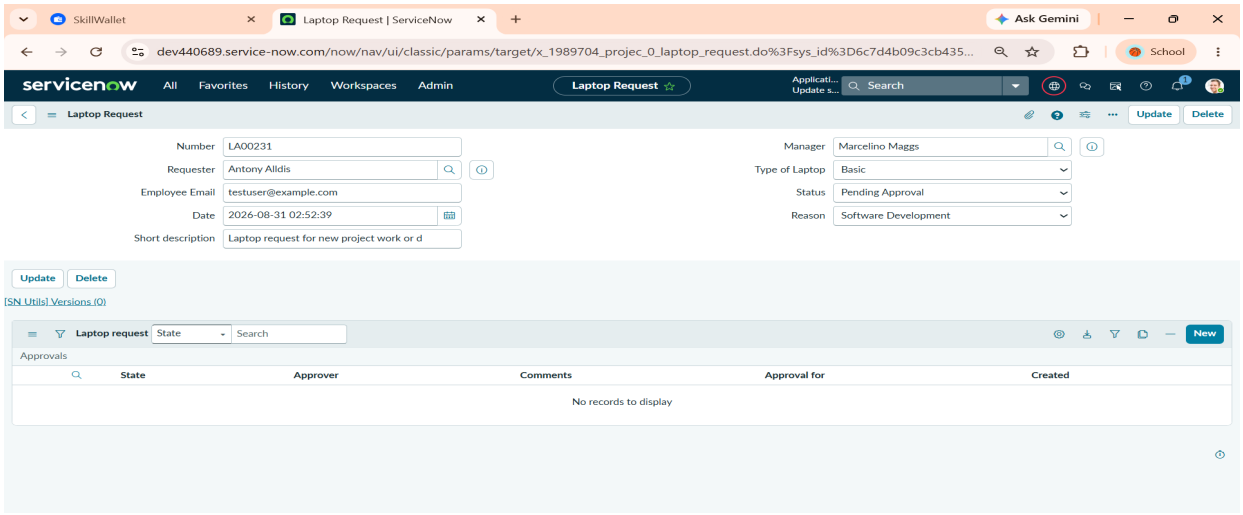


Figure 3.3.4 – Generated Laptop Request Record Verification in Target Table

4. Phase 3: Workflow Development & Execution

ServiceNow Flow Designer offers a codeless environment to automate end-to-end business logic and approval governance. For the Employee Laptop Request process, Flow Designer intercepts newly created records, initializes standard tracking states, requests dynamic manager approvals, and handles conditional decision branching. Automated email notifications and status transitions remove manual intervention, making sure of complete process transparency from initial intake to final decision.

1. Accessing Flow Designer

Navigate to ServiceNow's process automation interface to create and manage automated workflows and subflows across custom application tables.

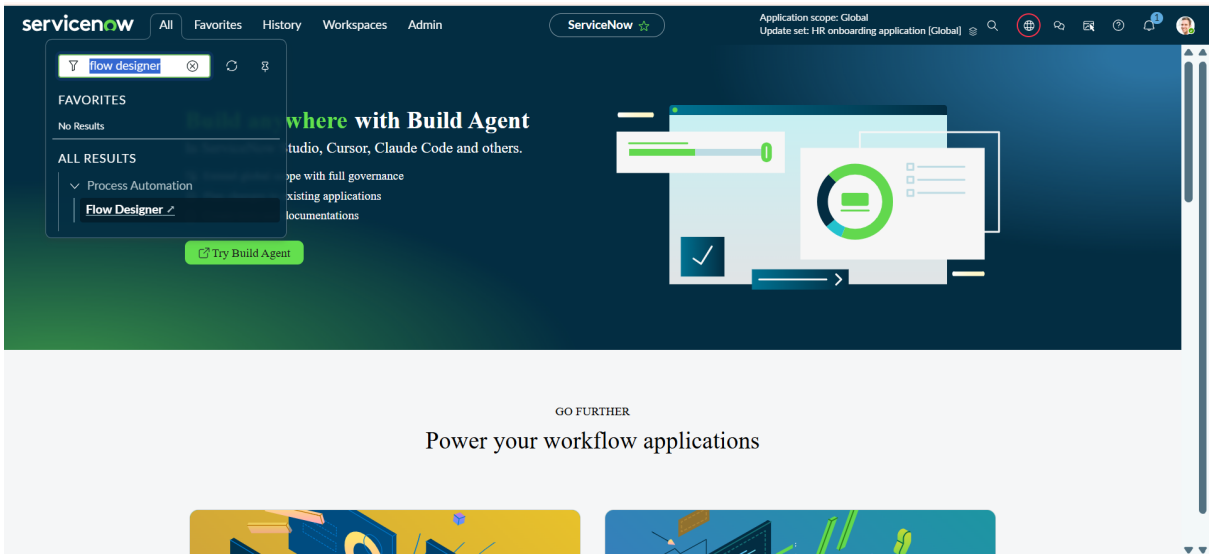


Figure 4.1 – Accessing Flow Designer under Process Automation Module

2. Configuring the Flow Trigger

Specify the entry condition that executes the workflow. The flow triggers automatically whenever a new record is inserted into the custom Laptop Request table (x_1989704_projec_0_laptop_request), immediately processing the submitted catalog request.

Trigger Settings:

- **Trigger:** Created
- **Table:** Laptop Request [x_1989704_projec_0_laptop_request]

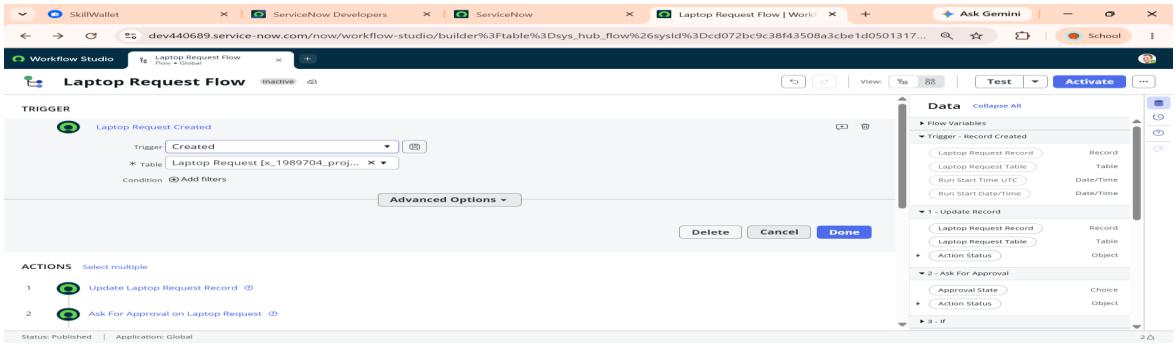


Figure 4.2 – Configuring Flow Record Creation Trigger on Laptop Request Table

3. Initializing Record State (Default Status)

Update the newly created record immediately upon flow execution to standardize its lifecycle state and prevent unmanaged submissions from advancing without authorization.

Action Inputs:

- **Action:** Update Record
- **Record:** Trigger -> Laptop Request Record
- **Fields:** Status → Pending Approval

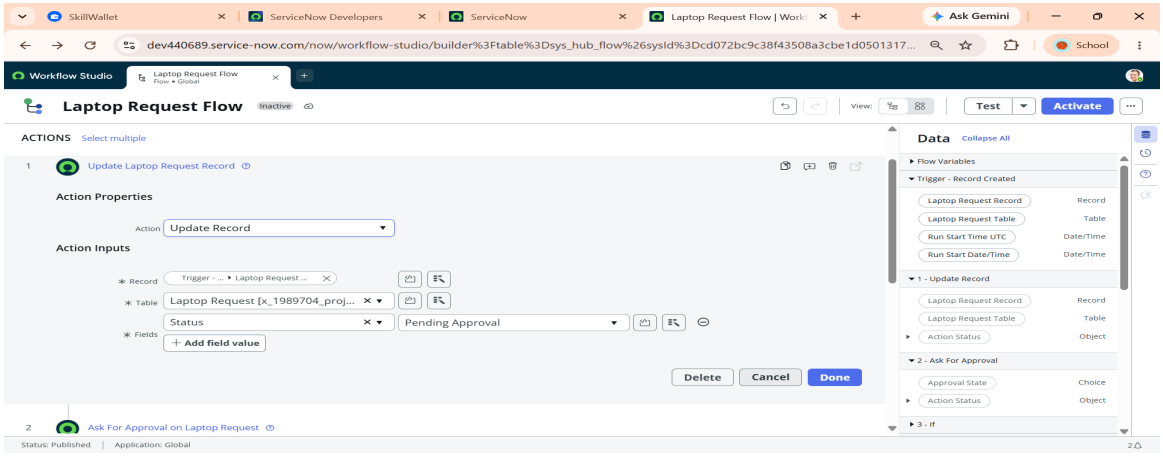


Figure 4.3 – Setting Default Status to Pending Approval via Update Record Action

4. Manager Approval Routing

Trigger an approval request using the platform's core approval framework. The flow dynamically extracts the manager referenced on the trigger record, generates an entry in sysapproval_approver, and pauses workflow execution until the manager acts.

Action Inputs:

- **Action:** Ask For Approval
- **Record:** Trigger -> Laptop Request Record

- **Rules:** Approve when Anyone approves (Trigger -> Requester's Manager) **OR** Reject when Anyone rejects (Trigger -> Requester's Manager)

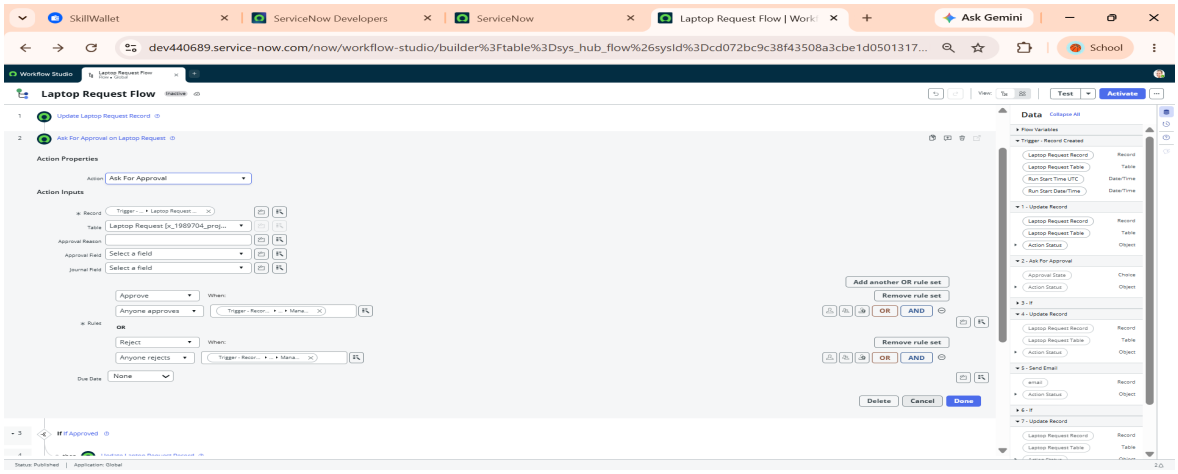


Figure 4.4 – Configuring Ask For Approval Action with Dynamic Manager Mapping

5. Approval Decision Branching (If Approved)

Build conditional logic to evaluate the manager's response. When the approval state is evaluated as Approved, the flow branches into positive fulfillment steps.

Logic Condition:

- **Condition Label:** If Approved
- **Condition:** 2 - Ask For Approval -> Approval State is Approved

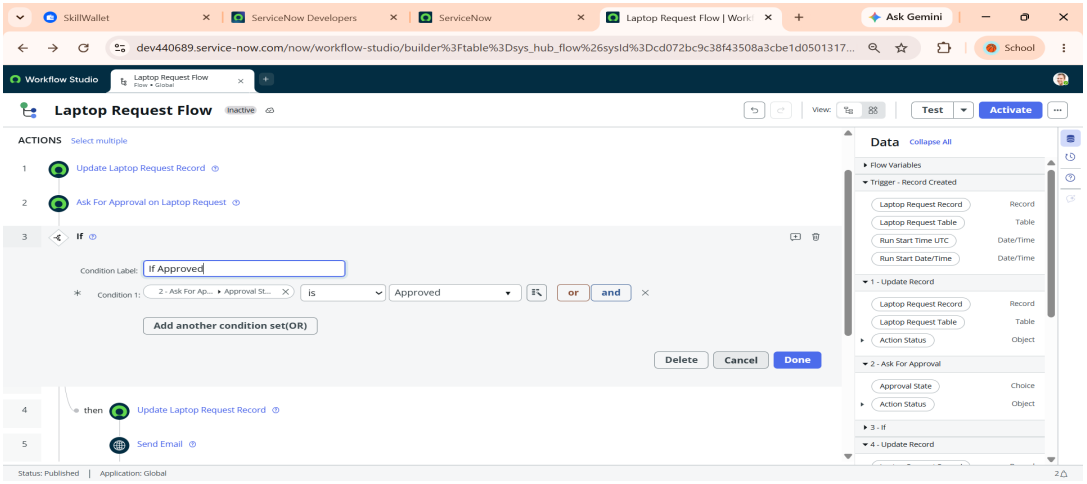


Figure 4.5 – Evaluating Approval Decision Condition (If Approved Flow Logic) 6. Updating Status to Approved

6. Updating Status to Approved

Execute a targeted update on the source record within the approved branch, moving the request from pending governance into the approved state.

Action Inputs:

- **Action:** Update Record
- **Record:** Trigger -> Laptop Request Record
- **Fields:** Status → Approved

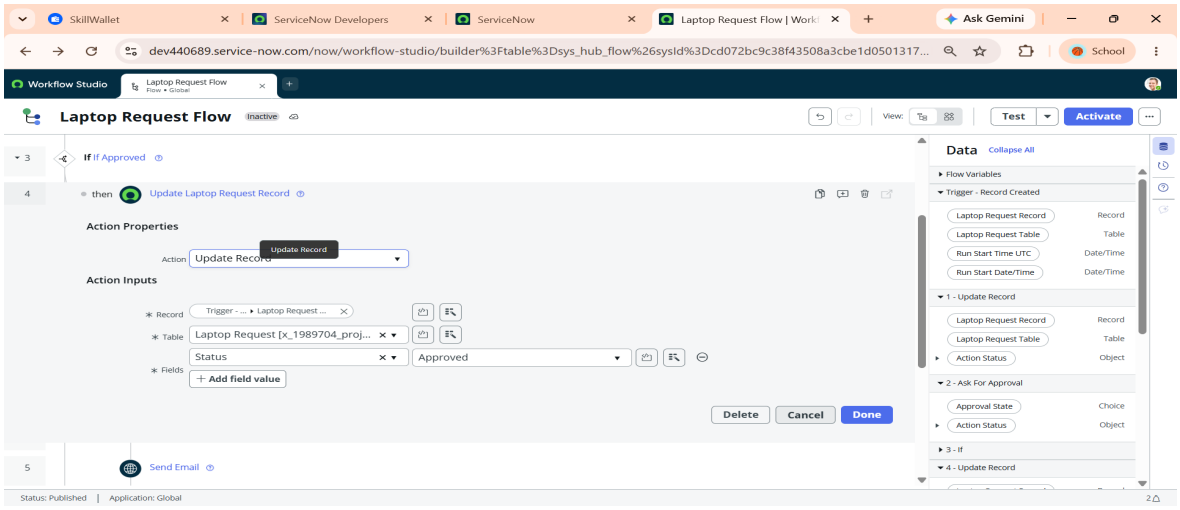


Figure 4.6 – Updating Request Status to Approved upon Manager Authorization

7. Dispatching Approval Email Notification

Notify the requester that their laptop request has been authorized by sending an automated email notification containing dynamic record data.

Action Inputs:

- **Action:** Send Email
- **To:** Trigger -> Laptop Request Record -> Employee Email
- **Subject:** Laptop Request Approved
- **Body:** Your laptop request has been approved.

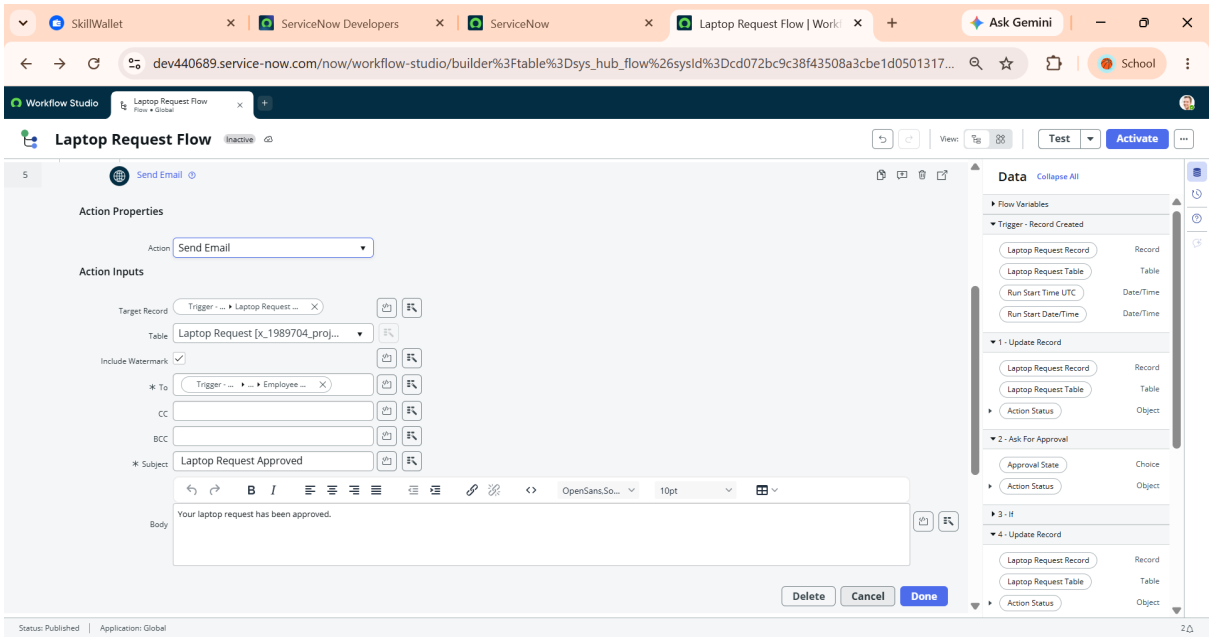


Figure 4.7 – Configuring Automated Approval Notification Email to Requester

8. Rejection Decision Branching (If Rejected)

Build parallel decision logic to handle scenarios where the manager rejects the submission, ensuring controlled termination of the request.

Logic Condition:

- **Condition Label:** If Rejected
- **Condition:** 2 - Ask For Approval -> Approval State is Rejected

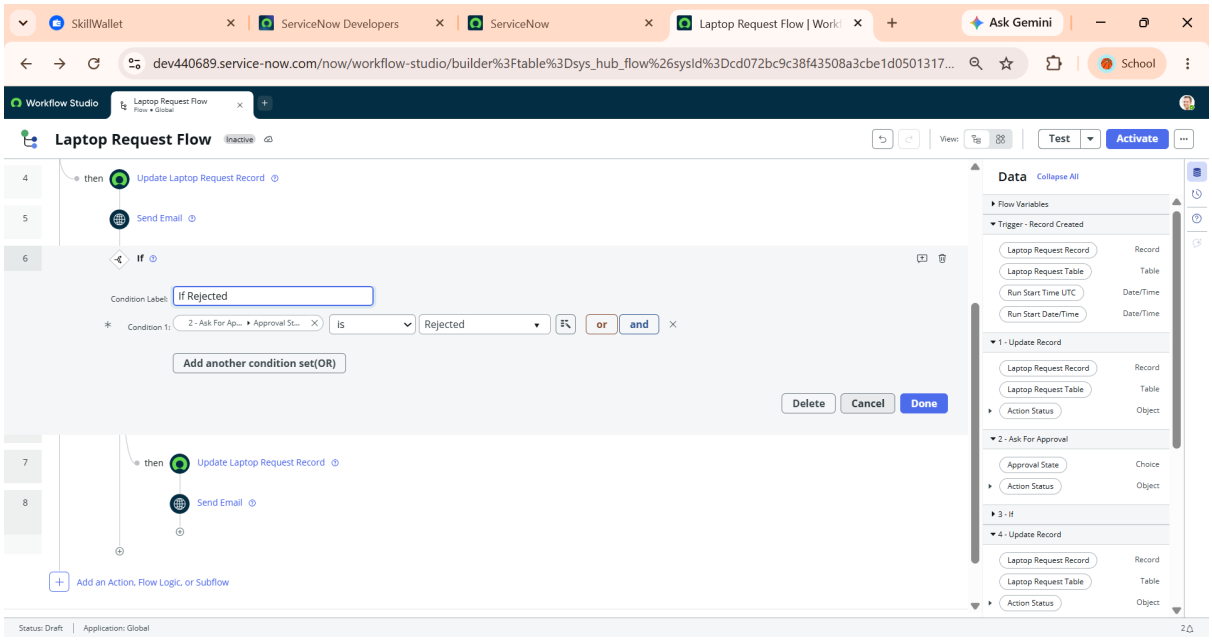


Figure 4.8 – Configuring Rejection Flow Logic Branch for Denied Requests

9. Updating Status to Rejected

Update the parent record's state within the rejection branch to close out the request lifecycle and prevent further fulfillment tasks.

Action Inputs:

- **Action:** Update Record
- **Record:** Trigger -> Laptop Request Record
- **Fields:** Status → Rejected

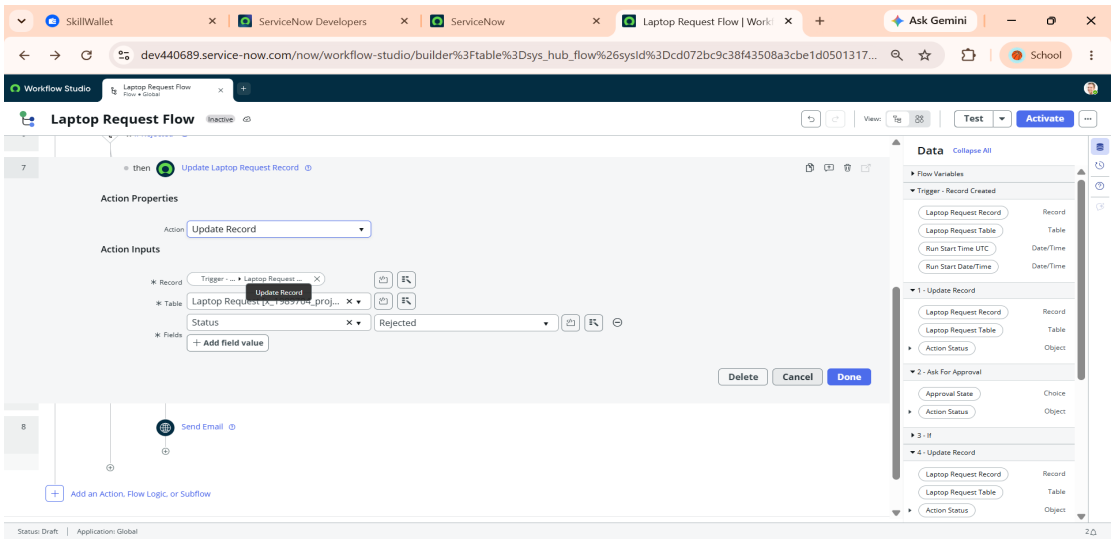


Figure 4.9 – Updating Request Status to Rejected within Rejection Path

10. Dispatching Rejection Email Notification

Send clear, automated communication to the employee explaining that the request was declined, formally closing out the automated lifecycle.

Action Inputs:

- **Action:** Send Email
- **To:** Trigger -> Laptop Request Record -> Employee Email
- **Subject:** Laptop Request Rejected.
- **Body:** Your laptop request has been rejected.

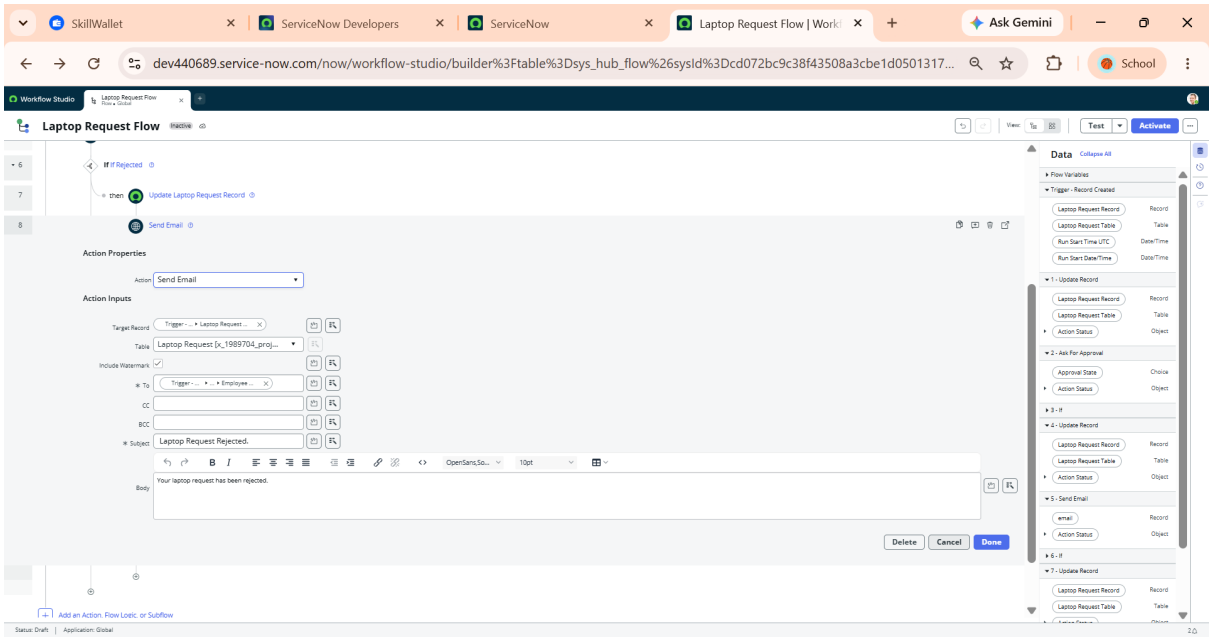


Figure 4.10 – Configuring Automated Rejection Email Notification to Requester

11. End-to-End Laptop Request Flow Overview

Examine the complete, consolidated workflow structure in Flow Designer / Workflow Studio canvas. This visual hierarchy displays the full lifecycle: trigger execution on record creation, initial status initialization to *Pending Approval*, dynamic manager approval routing via Ask For Approval, and conditional

branching for both *If Approved* and *If Rejected* execution paths (including target record status updates and employee email notifications).

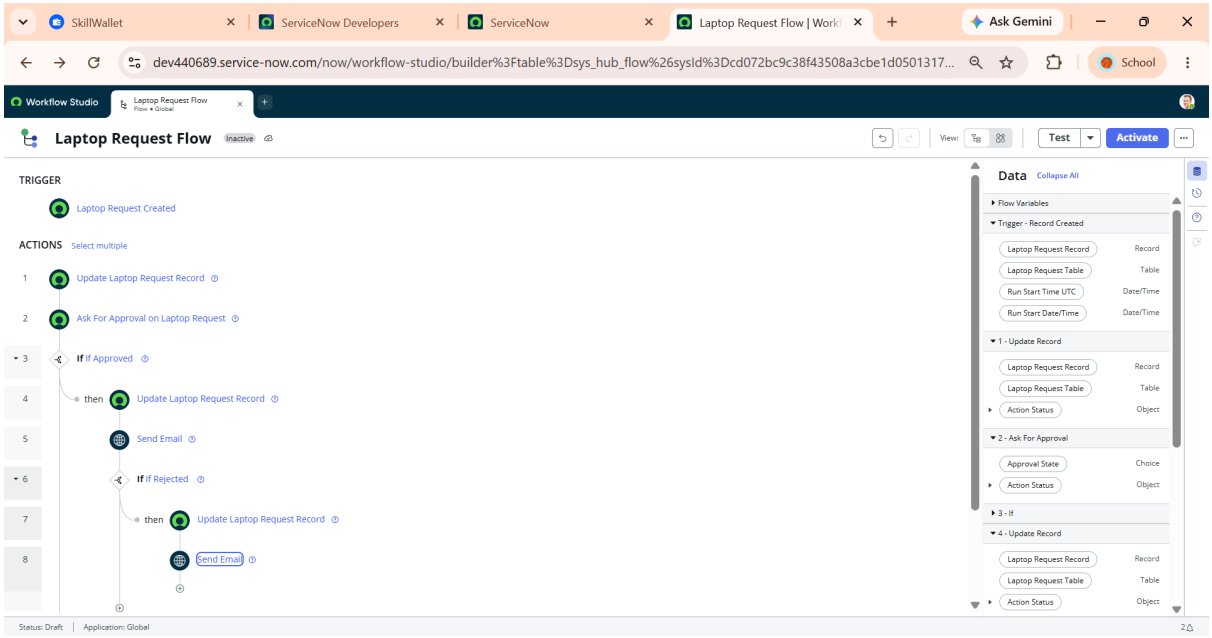


Figure 4.11 – Consolidated Laptop Request Flow Architecture and Execution Logic

5. Phase 4: Business Rule Creation, Security & Validation

Business Rule Creation, Security & Validation concentrates on policy enforcement, data integrity, and role-based access control within the Employee Laptop Request Management System. In this phase, server-side Business Rules are configured to prevent duplicate laptop submissions by validating active requests against existing employee records prior to database insertion. Additionally, Access Control Lists (ACLs) and security rules are implemented to restrict record permissions: employees can securely create and manage their own submissions, while managers are granted read-only visibility over their direct reports' requests. All validation scripts, security constraints, and policy controls are developed and captured systematically within the HR Onboarding Application update set to maintain environment consistency and compliance.

5.1. Business Rule Creation

Server-side Business Rules in ServiceNow enforce core business logic, data integrity, and organizational policies prior to database transactions. To prevent duplicate laptop provisioning and enforce company policy limiting employees to one laptop request per calendar year, a before insert Business Rule intercepts incoming submissions. The rule executes a backend query on historical records for the same requester within the active year boundary; if an existing request is detected, the database transaction is aborted and a clear error notification is displayed to the user.

1. Navigating to Business Rules Module

Open the Business Rules definition interface under System Definition to create server-side validation logic that runs when records are queried, inserted, updated, or deleted.

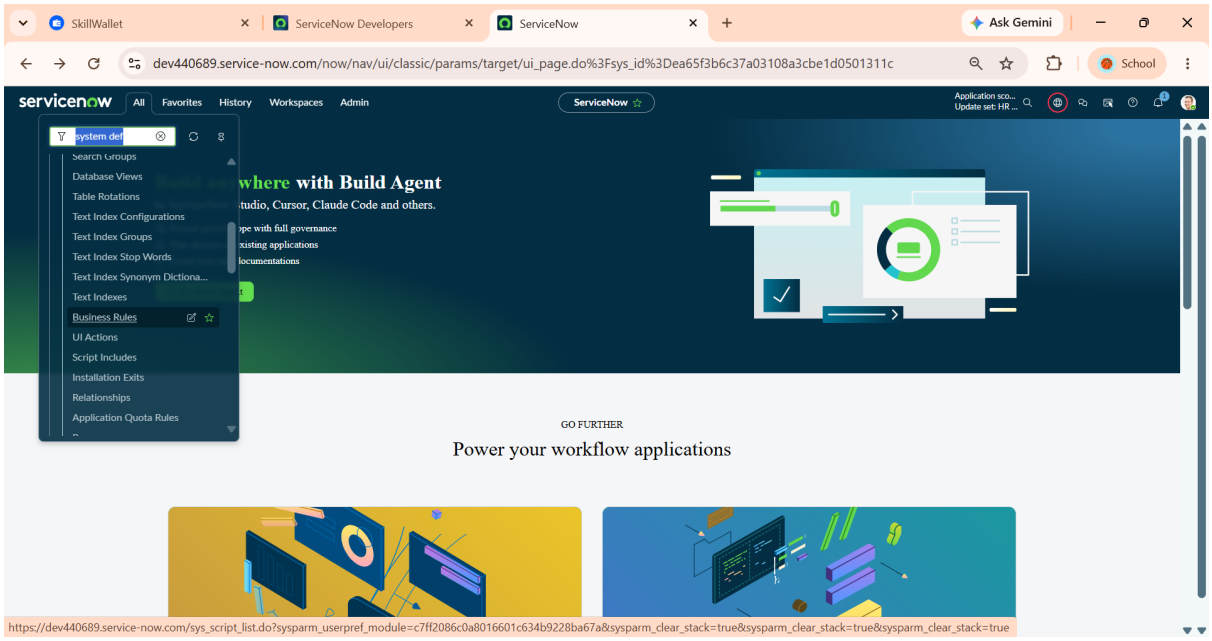


Figure 5.1.1 – Navigating to Business Rules under System Definition

2. Configuring Business Rule Header and Target Table

Specify the name, target scoped table, and execution options for the Business Rule. Enabling the Advanced checkbox unlocks execution timing controls, filter conditions, and custom script editors.

Key Fields:

- **Name:** Laptop request BR
- **Table:** Laptop Request [x_1989704_projec_0_laptop_request]
- **Active:** Enabled (Checked)
- **Advanced:** Enabled (Checked)

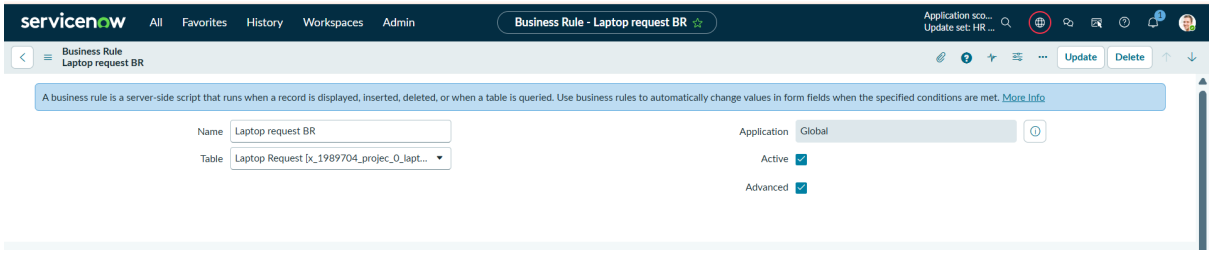


Figure 5.1.2 – Defining Business Rule Name, Target Table, and Advanced Configuration

3. Setting Execution Timing and Database Triggers

Specify the execution phase for the rule. Setting **When** to before and selecting Insert ensures validation evaluates before the record is saved to the database, allowing the engine to abort the transaction if policies are violated without creating orphaned data.

Timing Settings:

- **When:** before
- **Order:** 100
- **Database Operations:** Insert (Checked)

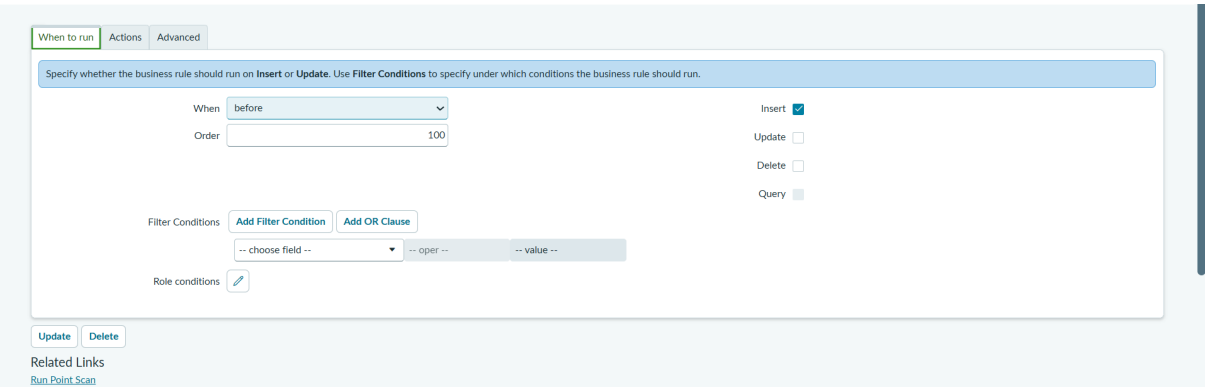


Figure 5.1.3 – Configuring When to Run Settings for Before Insert Execution

4. Implementing Validation Script via GlideRecord

Write server-side logic in the **Advanced** tab using ServiceNow JavaScript APIs (GlideSystem and GlideRecord). The script calculates current year bounds (gs.beginningOfThisYear() / gs.endOfThisYear()), queries the request table for prior submissions by the same employee within that range, displays an error message via gs.addErrorMessage(), and halts database insertion using current.setAbortAction(true).

Script Implementation:

(function executeRule(current, previous /*null when async*/) {

```
    var startOfYear = gs.beginningOfYear();
    var endOfYear = gs.endOfYear();
    // Check for existing request by same requester in the same year
    var gr = new GlideRecord('u_laptop_request_form');
    gr.addQuery('u_requester', current.u_requester);
    gr.addQuery('sys_created_on', '>=', startOfYear);
    gr.addQuery('sys_created_on', '<=', endOfYear);
    gr.query();
    if (gr.next()) {
        gs.addErrorMessage('You have already submitted a laptop request for this year.');
        current.setAbortAction(true);
    }
})(current, previous);
```

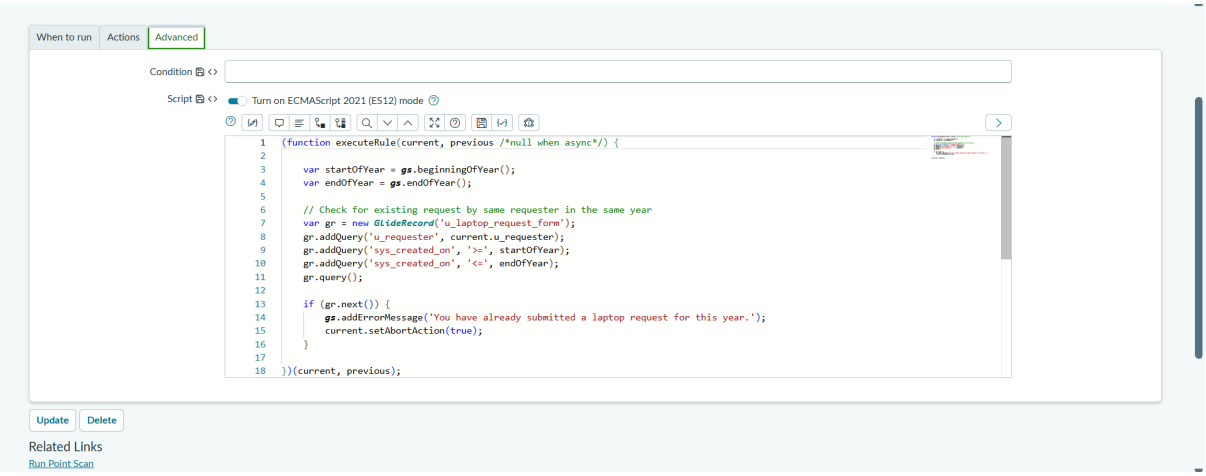


Figure 5.1,4 – Business Rule Advanced Script for Annual Request Policy Enforcement

5.2. Role Creation

Role-Based Access Control (RBAC) in ServiceNow establishes granular security boundaries across custom application data and modules. By defining dedicated roles, the platform restricts record creation, modification, and visibility according to organizational responsibilities. For the Employee Laptop Request Management System, distinct roles are established: it_manager to grant read-only visibility for approvers, and it_employee to authorize end users to create, submit, and track their laptop requests securely.

1. Navigating to Roles Module

Open the centralized user role repository under the System Security application to configure and register new user roles for the platform.

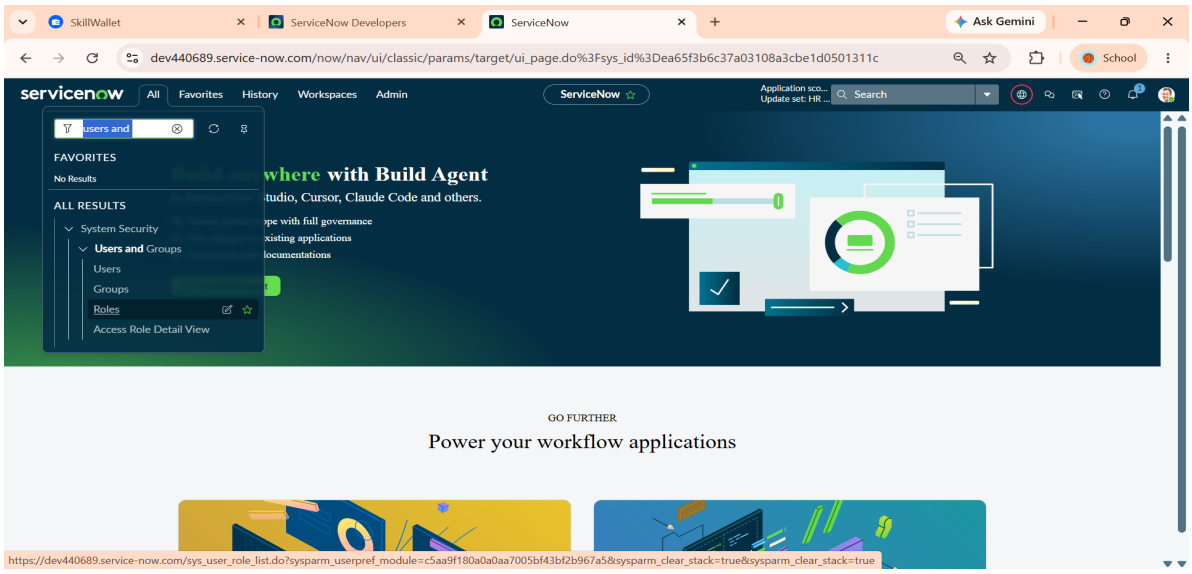


Figure 5.2.1 – Navigating to Roles under System Security Users and Groups

2. Creating the IT Manager Role (it_manager)

Create a dedicated role tailored for managers and approvers. This role provides the baseline access required to view submitted requests, review justification details, and evaluate approvals without granting broad edit permissions over underlying table records.

Role Settings:

- **Name:** it_manager
- **Application:** Global
- **Description:** View-only access for managers
- **Elevated privilege:** Disabled (Unchecked)

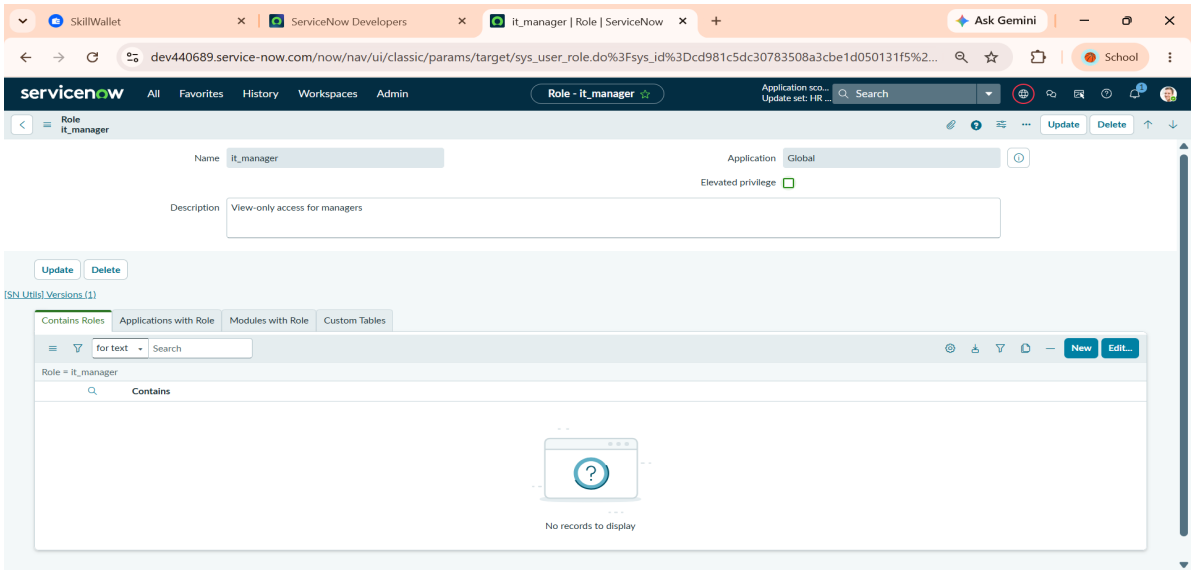


Figure 5.2.2 – Defining the it_manager Role Configuration for Manager Read Access

3. Creating the IT Employee Role (it_employee)

Create the end-user role enabling standard employees to interact with the service catalog item, submit new laptop intake requests, and track the real-time status of their own records.

Role Settings:

- **Name:** it_employee
- **Application:** Global
- **Description:** Create and manage laptop requests
- **Elevated privilege:** Disabled (Unchecked)

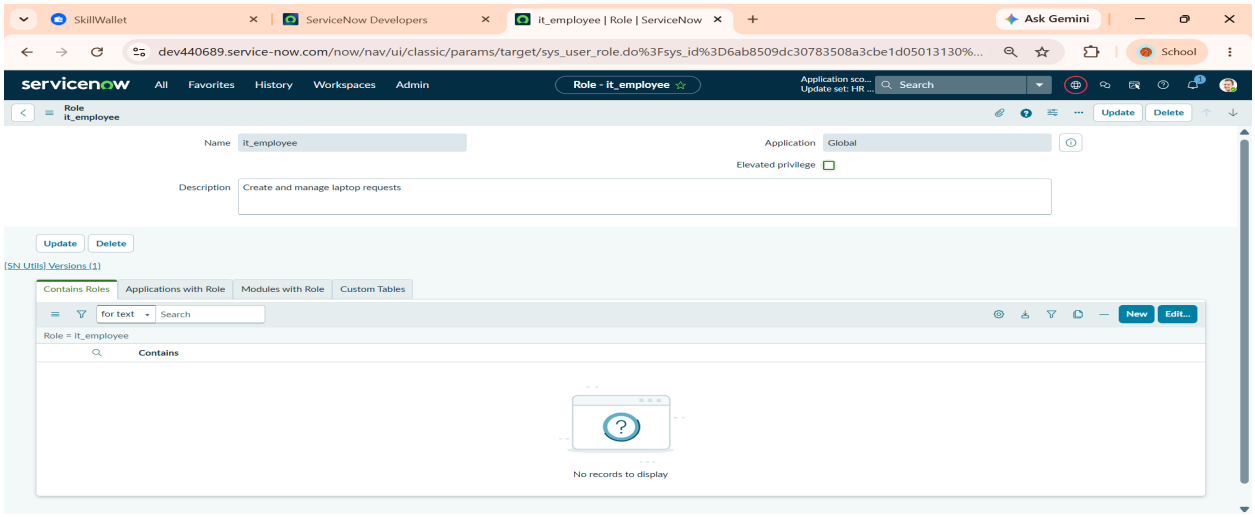


Figure 5.2.3– Defining the it_employee Role Configuration for Request Creation and Management

5.3. Group Creation & User Mapping

In ServiceNow, managing permissions via User Groups rather than individual user assignments is an industry best practice for scalable security administration. Groups simplify access governance by associating specific business roles (it_manager and it_employee) directly with functional teams. Every member added to a group automatically inherits its assigned roles and permissions, ensuring secure access boundaries for both management oversight and employee request submissions.

1. Navigating to the Groups Module

Open the centralized User Groups management interface to configure administrative teams and define role inheritance structures.

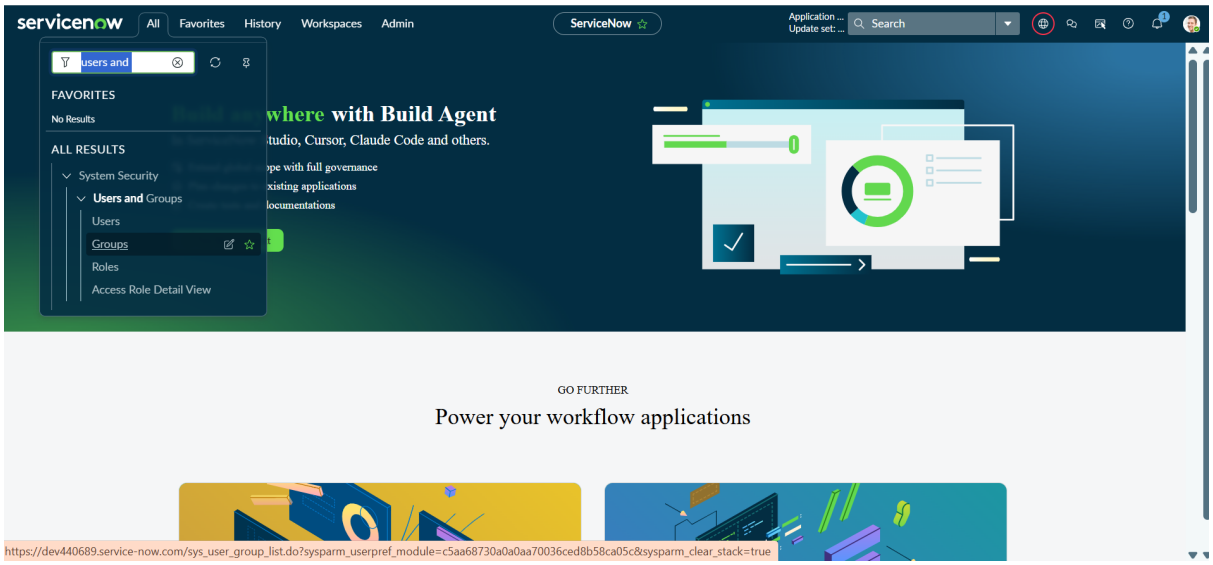


Figure 5.3.1– Navigating to Groups under System Security Users and Groups

2. Creating the IT Managers Group

Set up a dedicated group container for manager personas responsible for reviewing, tracking, and approving laptop request submissions.

Group Details:

- **Name:** IT Managers

- **Description:** who approves the request

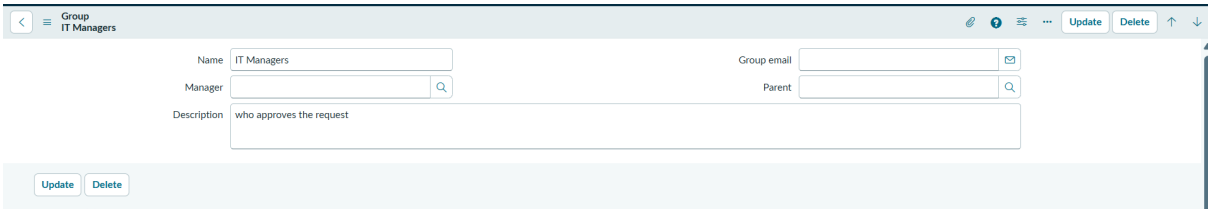


Figure 5.3.2 – Defining IT Managers Group Header and Description

3. Assigning the it_manager Role to IT Managers Group

Associate the it_manager role with the IT Managers group to automatically cascade view and approval permissions down to all group members.

Role Configuration: it_manager (Inherits: true).

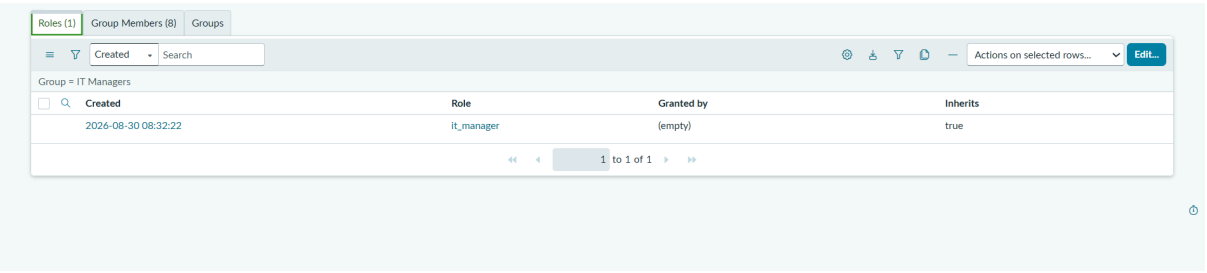


Figure 5.3.3 – Assigning the it_manager Role to the IT Managers Group

4. Adding Members to the IT Managers Group

Add manager user accounts to the IT Managers group to grant them active operational access.

Group Members (8 Users): Allie Pumphrey, Alfonso Griglen, Alejandro Mascall, Aileen Mottern, Allan Schwantd, Abel Tuter, Alissa Mountjoy, Adela Cervantsz.

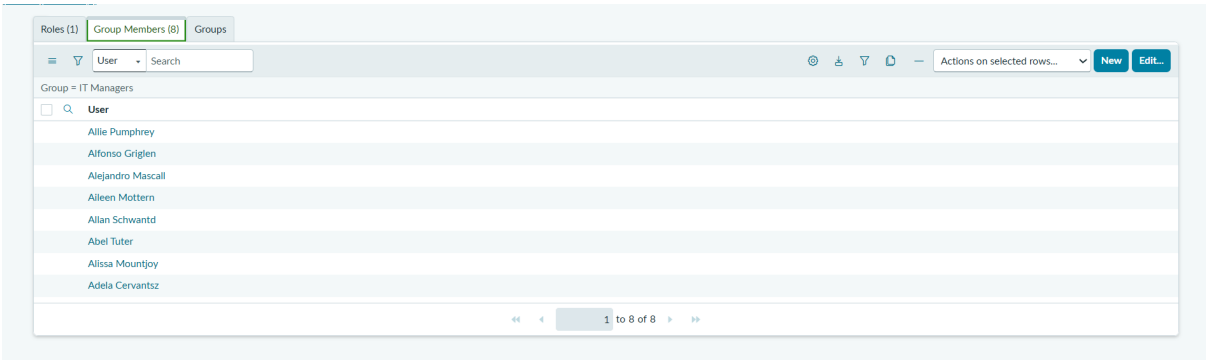


Figure 5.3.4 – Mapping Manager User Accounts to the IT Managers Group

5. Creating the IT Employees Group

Set up the baseline user group for standard organizational staff who submit intake requests through the Employee Service Catalog.

Group Details:

- **Name:** IT Employees
- **Description:** Employee members in it companies

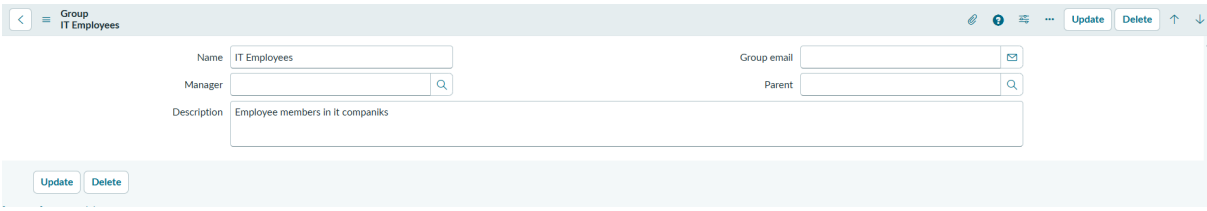


Figure 5.3.5 – Defining IT Employees Group Header and Configuration

6. Assigning the it_employee Role to IT Employees Group

Attach the it_employee role to the IT Employees group so all mapped employees inherit the permissions needed to create and track laptop requests.

Role Configuration: it_employee (Inherits: true).

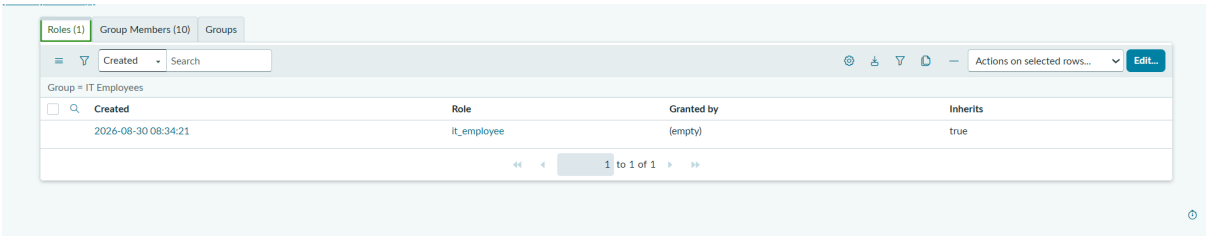


Figure 5.3.6 – Assigning the it_employee Role to the IT Employees Group

7. Adding Members to the IT Employees Group

Fill the IT Employees group with user profiles to finalize the end-user access baseline.

Group Members (10 Users): Adela Cervantsz, Alene Rabeck, Allan Schwantd, Alejandro Mascall, Alejandra Prenatt, Alissa Mountjoy, Abel Tuter, Abraham Lincoln, Aileen Mottern, Alfonso Griglen.

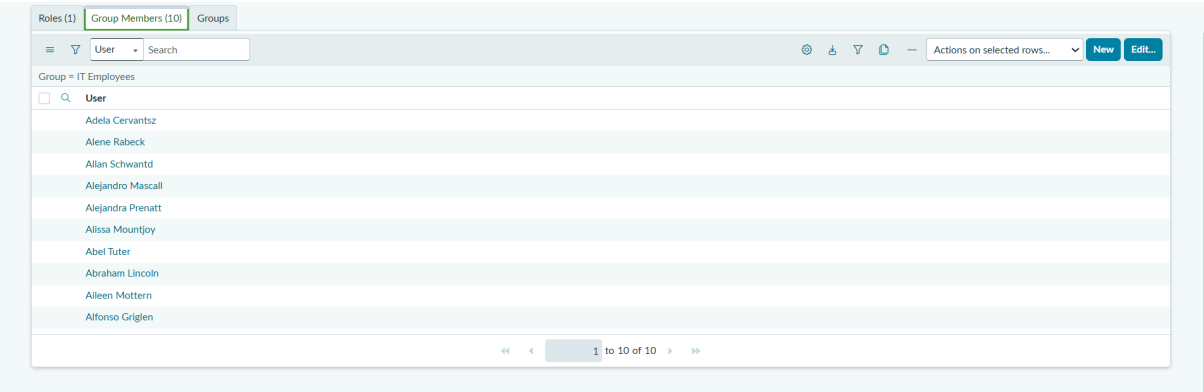


Figure 5.3.7 – Mapping Employee Accounts to the IT Employees Group

5.4. Access Control List (ACL) Configuration & Security Validation

Access Control Lists (ACLs) enforce table-level and field-level security rules directly within the ServiceNow database layer. To protect sensitive request records and enforce segregation of duties, table-level ACL rules (tablename.none) restrict operations strictly by user role. Administrators must first elevate their security privileges (security_admin) to create and modify ACLs. Under this security configuration, users with the it_manager role obtain read-only access to review submissions, while users with the it_employee role are authorized to create, read, and write laptop request records.

1. Configuring the Read ACL for Manager and Employee Roles

Create a baseline table-level read ACL rule bound to both it_manager and it_employee roles. This allows managers to review submitted requests in list/form views and enables employees to track the status of their own records.

Rule Settings:

- Type: record
- Operation: read
- Name: Laptop Request [x_1989704_projec_0_laptop_request] (None)
- Requires Role: it_manager, it_employee

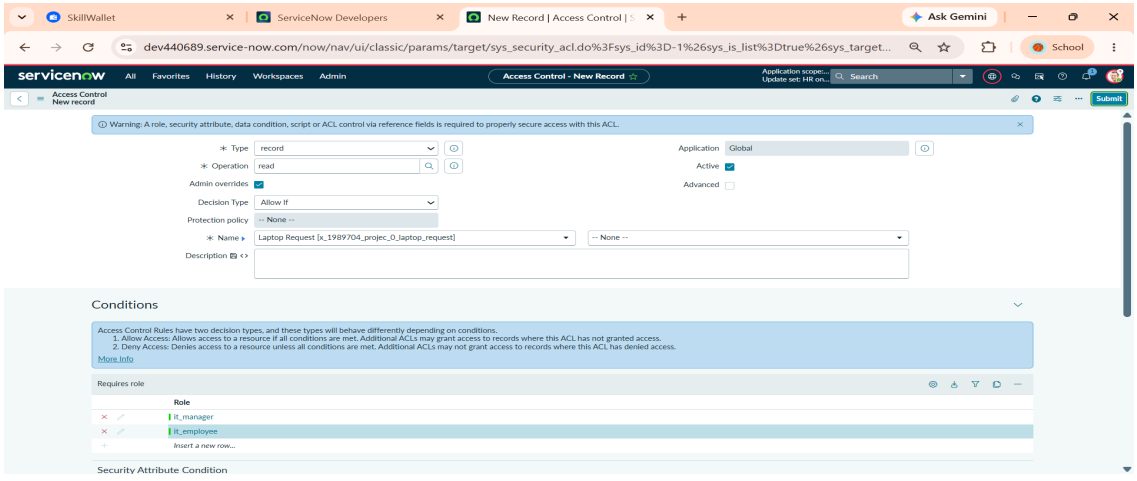


Figure 5.4.1 – Configuring Read Table-Level Access Control for it_manager and it_employee

2. Verifying Security Rule Impact for Read Operation

Confirm and apply the row-level security rule addition on the target scoped database table (x_1989704_projec_0_laptop_request) to enforce active read restrictions.

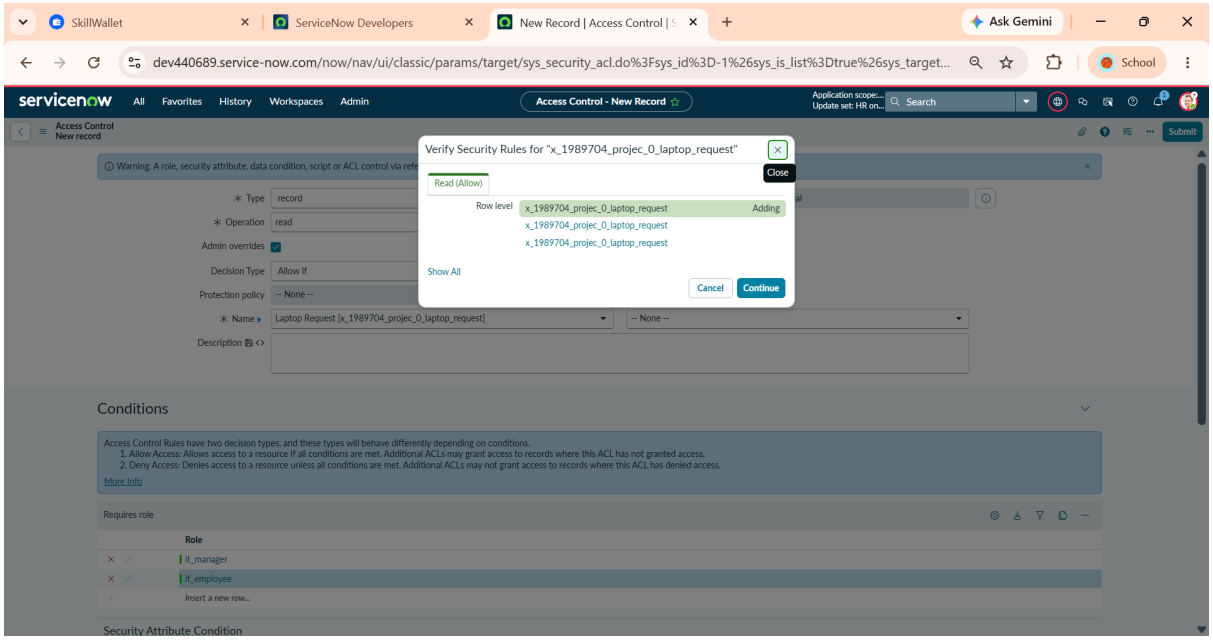


Figure 5.4.2– Verifying Row-Level Security Rules for Read Operation

3. Configuring the Create ACL for Employee Role

Set up a dedicated create operation ACL rule assigned exclusively to the it_employee role. This allows employees to initiate and log new laptop intake records while blocking managers and unauthorized personas from creating records directly in the backend.

Rule Settings:

- Type: record
- Operation: create
- Name: Laptop Request [x_1989704_projec_0_laptop_request] (None)
- Requires Role: it_employee

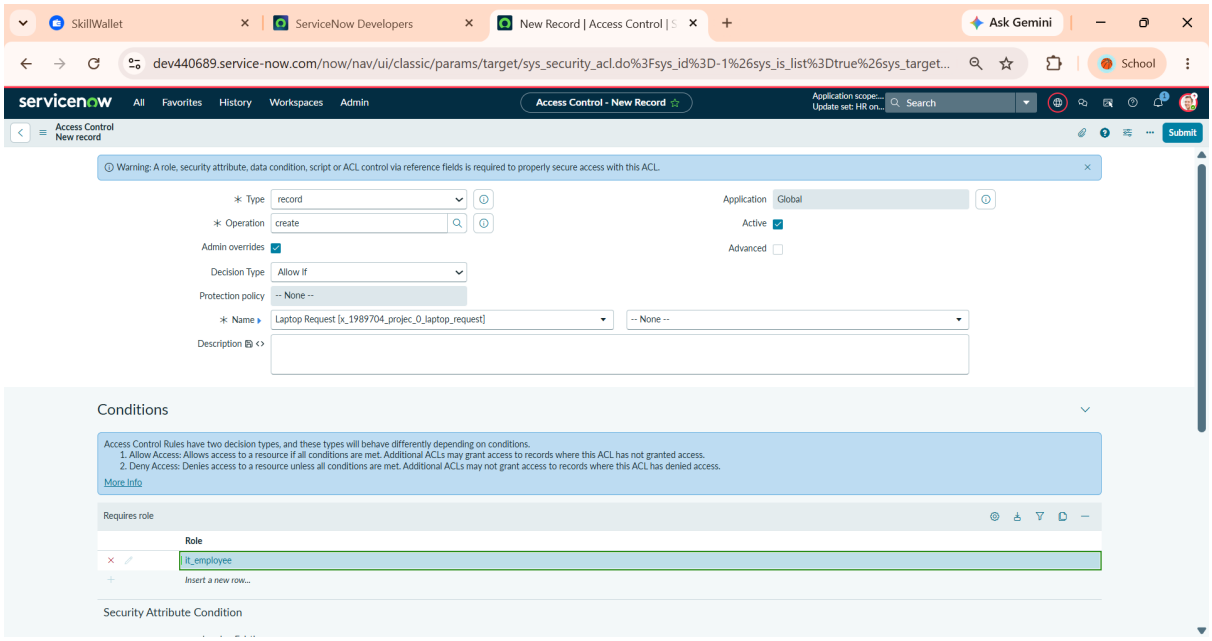


Figure 5.4.3 – Configuring Create Access Control Rule for it_employee Role

4. Verifying Security Rule Impact for Create Operation

Validate the newly registered create operation rule through the security confirmation prompt to activate record insertion rights for authorized employees.

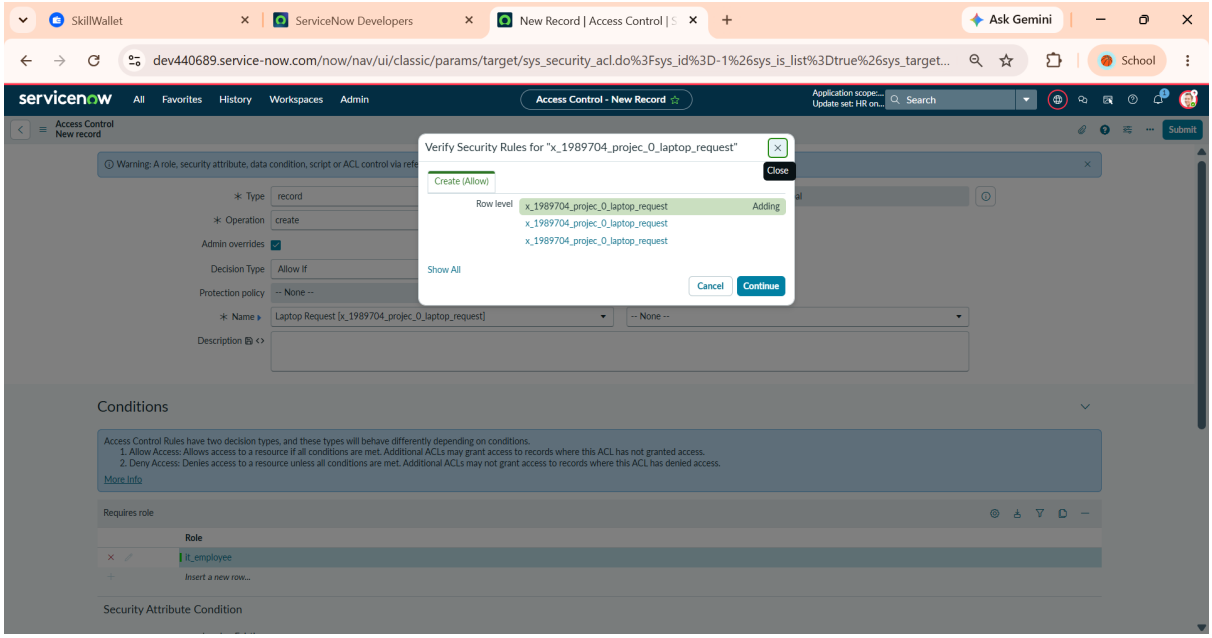


Figure 5.4.4 – Verifying Row-Level Security Rules for Create Operation

5. Configuring the Write ACL for Employee Role

Set up a write operation ACL restricted to the it_employee role. This ensures employees can modify and update their own request records, while managers remain restricted to read-only visibility without record editing rights.

Rule Settings:

- Type: record
- Operation: write
- Name: Laptop Request [x_1989704_projec_0_laptop_request] (None)
- Requires Role: it_employee

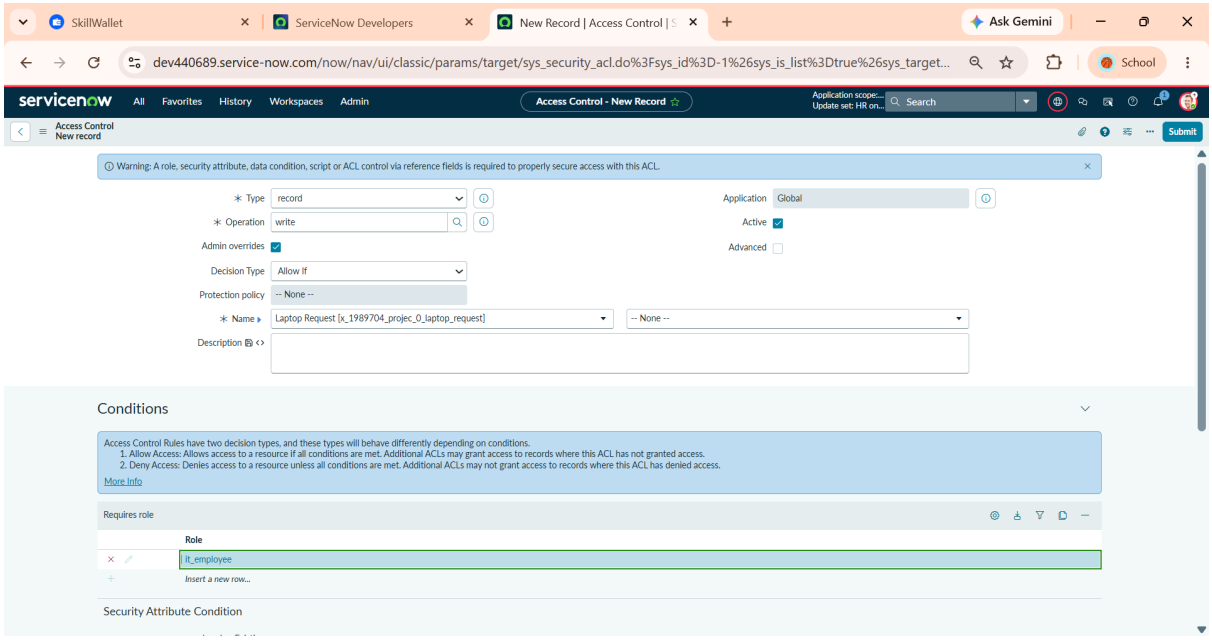


Figure 5.5.5– Configuring Write Access Control Rule for it_employee Role

6. Verifying Security Rule Impact for Write Operation

Finish the ACL setup by confirming the row-level write rule in the platform verification prompt, finalizing the full role-based CRUD security matrix.

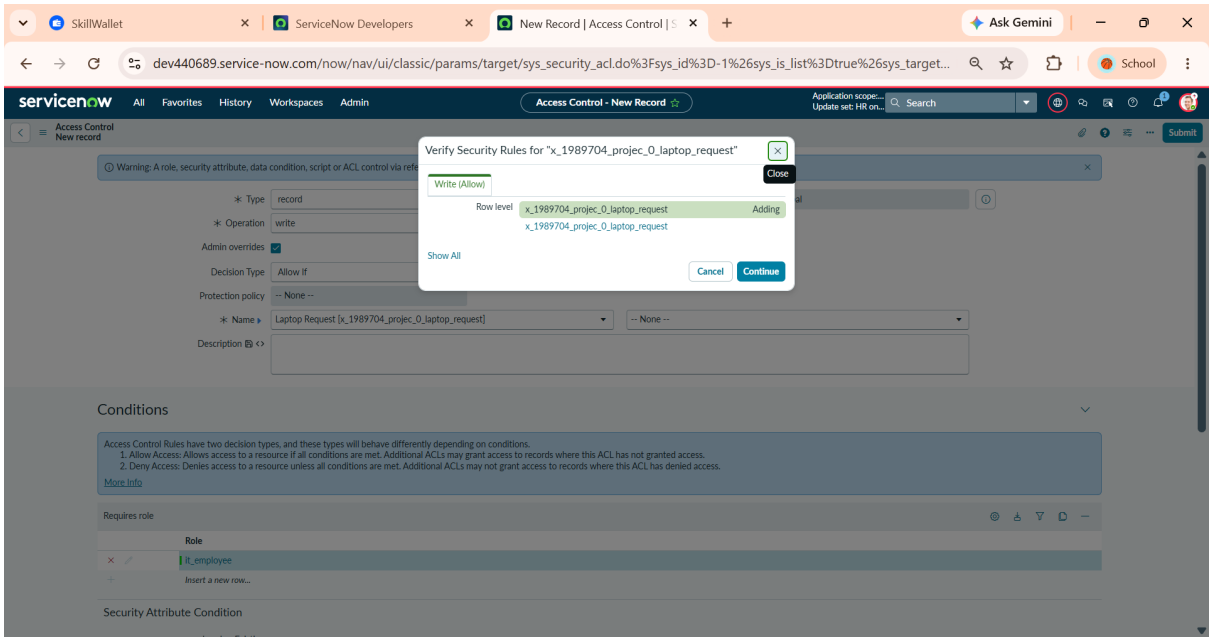


Figure 5.5.6– Verifying Row-Level Security Rules for Write Operation

Security & Functional Validation Matrix

Test Scenario	User Persona / Role	Target Operation	Expected Result	System Enforcement Mechanism
Employee creates request	Employee (it_employee)	Create	Allowed	Create ACL (sys_security_acl)
Employee edits request	Employee (it_employee)	Write	Allowed	Write ACL (sys_security_acl)
Duplicate annual request	Employee (it_employee)	Insert	Blocked	Business Rule (Laptop request BR)
Manager views request list	Manager (it_manager)	Read	Allowed	Read ACL (sys_security_acl)
Manager edits/creates record	Manager (it_manager)	Create / Write	Blocked	Role Restriction (No Create/Write ACLs)

6. Phase 5: Deployment, Final Presentation & Conclusion

Deployment, Final Presentation & Conclusion concentrates on the migration, end-to-end verification, and formal handover of the Employee Laptop Request Management System. In this closing phase, all platform artifacts—including custom table architectures, Record Producers, Flow Designer approval logic, server-side Business Rules, and role-based Access Control Lists (ACLs)—are finalized within the dedicated HR Onboarding Application update set. The completed update set is closed, exported, and retrieved into target testing and production instances for final deployment validation. Detailed technical documentation and administrator handovers are conducted to ensure seamless operational continuity and long-term maintainability across the ServiceNow platform.

6.1. Update Set Closure, Export & Deployment Migration

In ServiceNow application lifecycle management, closing and exporting an Update Set packages all tracked customizations—such as scoped tables, Record Producers, Flow Designer workflows, Business Rules, and Access Control Lists—into an immutable XML file. This XML payload serves as the migration vehicle to transport configurations across instances (e.g., from Development to Test/Production) without manual re-creation. Once the development state transitions to *Complete*, administrators export the payload, upload it into the target instance, preview it to resolve schema conflicts, and commit it to finalize deployment.

1. Navigating to Local Update Sets

Open the local update set administrative module to review the development container housing all customized application components.

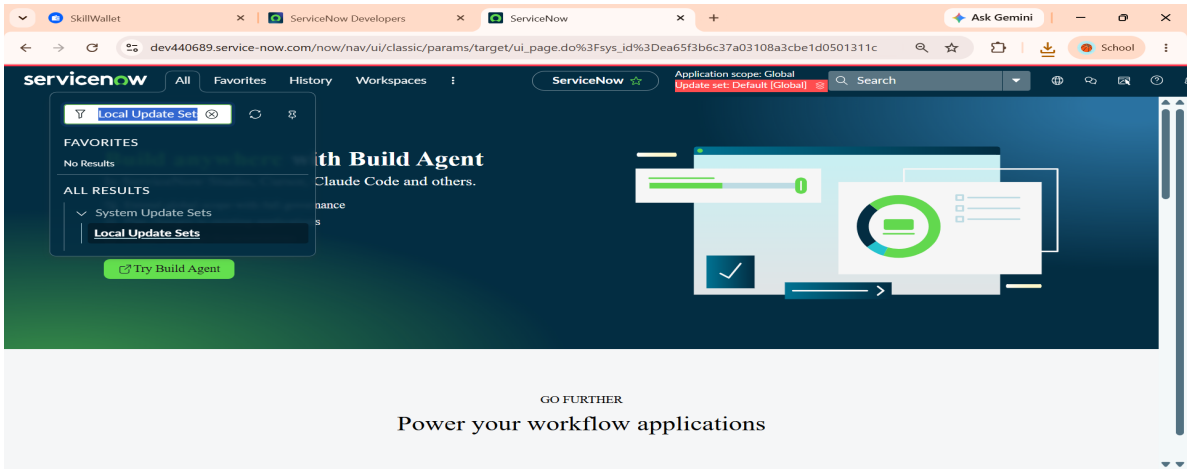


Figure 6.1 – Accessing Local Update Sets under System Update Sets

2. Closing the Update Set (State → Complete)

Change the operational state from *In Progress* to *Complete*. Marking an update set complete locks the container from receiving further modifications and unlocks migration-related actions such as export and remote retrieval.

Configuration:

- **Name:** HR onboarding application
- **State:** Complete

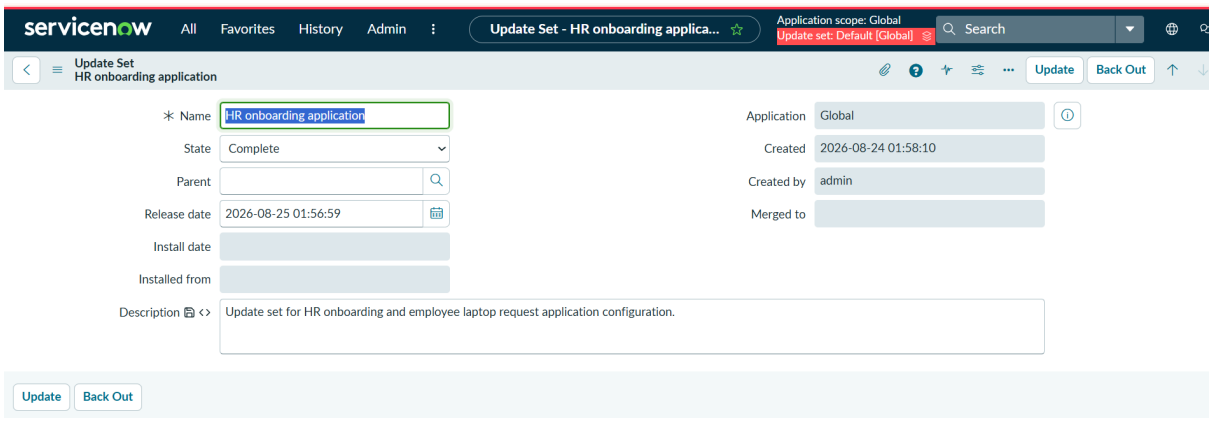


Figure 6.2 – Setting HR Onboarding Application Update Set State to Complete

3. Executing Export to XML via Related Links

Run the built-in XML generator to bundle all tracked configuration records (sys_update_xml entries) into an exportable file payload.

Action: Click the **Export to XML** action located under the **Related Links** section of the completed update set form.

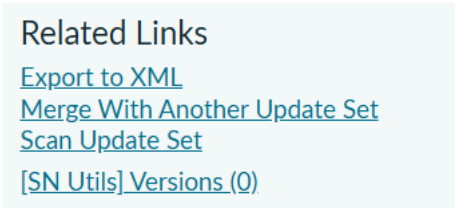


Figure 6.3 – Selecting Export to XML Option under Form Related Links

4. Downloading the Update Set Payload File

Save the generated XML payload (sys_remote_update_set_<sys_id>.xml) to the local file system. This packaged file is now ready to be imported into target test or production instances for preview, conflict resolution, and final commitment.

Payload Verification: Displays successful browser download of sys_remote_update_set_3755e8d5c38783508a3cbe1d05013164.xml containing the 31 tracked customer updates.

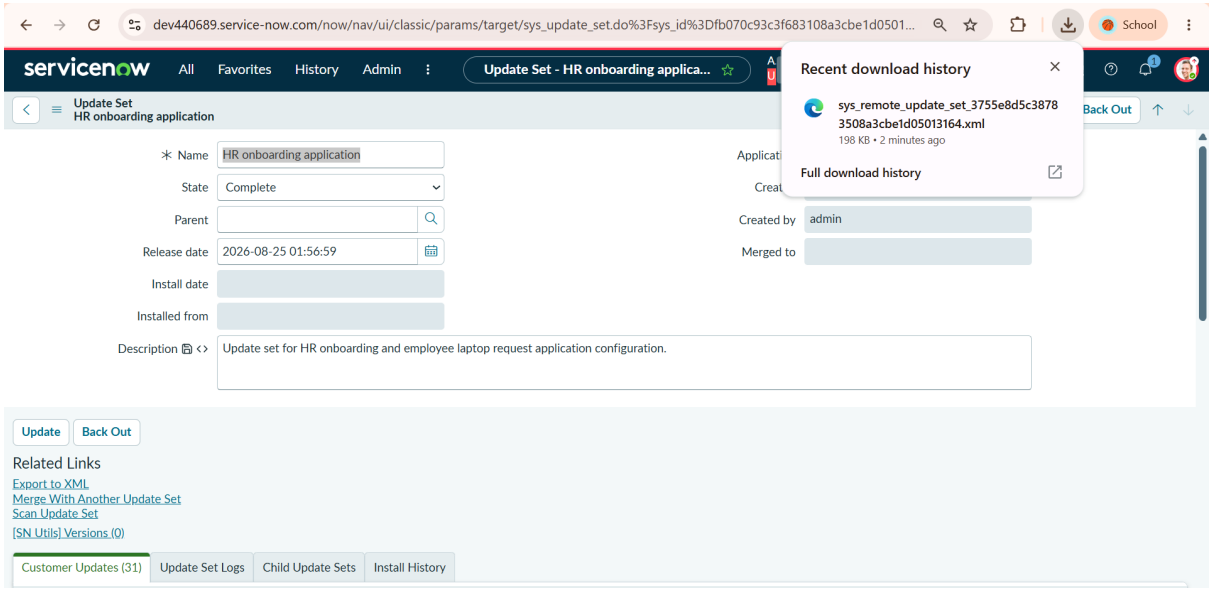


Figure 6.4 – Verified Local Download of Remote Update Set XML Migration File

Phase 6: Conclusion

The Employee Laptop Request Management System successfully provides a secure, automated, and scalable solution for managing laptop requests in a modern IT organization. By building a solid data architecture, creating easy-to-use intake mechanisms via Service Catalog Record Producers, and orchestrating end-to-end approvals through Flow Designer, the implementation removes manual handoffs and simplifies hardware provisioning. Server-side Business Rules and table-level Access Control Lists (ACLs) enforce strict annual request policies and role-based data governance, ensuring employees manage requests securely while managers retain dedicated approval oversight. Packaged and migrated through structured Update Sets, the system establishes a standardized, audit-ready framework that accelerates onboarding efficiency and scales smoothly with organizational growth.